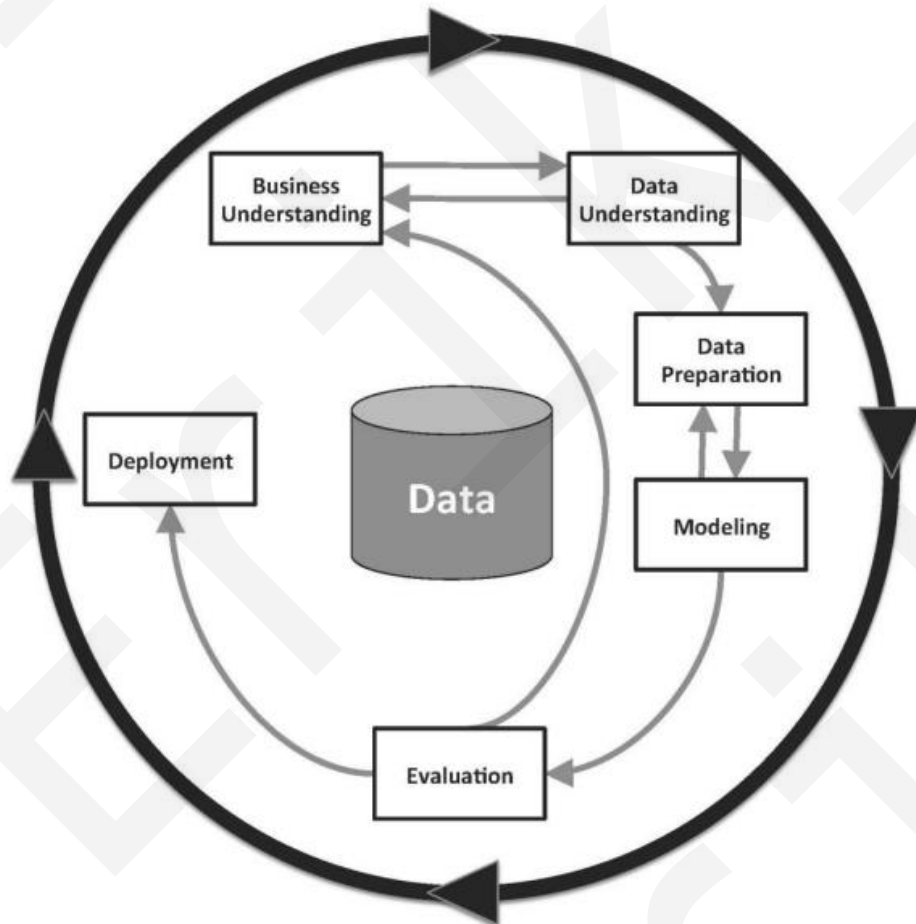


第一章：引言：机器学习及数据分析预测的基本概念



One of the most commonly used processes for predictive data analytics projects is the *Cross Industry Standard Process for Data Mining* (CRISP-DM) 预测数据分析项目最常用的流程之一是**数据挖掘的跨行业标准流程** 机器学习技术会自动从一组历史示例中学习一组**描述性特征与目标特征**之间的关系。

机器学习是一个病态的问题：

Generalize

Inductive bias

Underfitting

Overfitting

1. 什么是预测数据分析？

预测数据分析是数据分析的一个子领域，专注于构建模型，这些模型可以根据从历史数据中提取的见解进行预测。为了构建这些模型，我们使用机器学习算法从数据集中提取模式。

2. 什么是监督式机器学习？

监督式机器学习技术会自动从一组历史实例中学习一组描述性特征与目标特征之间的关系。监督式机器学习是机器学习的一个子领域。机器学习被定义为从数据中提取模式的自动化过程。

3. 机器学习通常被称为一个病态的问题。这是什么意思？

机器学习算法本质上是搜索一组描述性特征和目标特征之间存在的所有可能模式，以找到与所用训练数据一致的最佳模型。**可以找到与给定训练集一致多个模型（即与所有训练实例一致）**。出于这个原因，归纳机器学习被称为一个病态的问题，因为训练数据中**通常没有足够的信息**来选择单个最佳模型。归纳机器学习算法必须以某种方式选择一种可用的模型作为最佳模型。

5. 归纳偏置一词是什么意思？

学习算法的归纳偏差

- a. 是关于我们试图建模的真实函数是什么样子的一组假设；
- b. 定义学习算法在学习时考虑的一组假设；
- c. 引导学习算法优先选择一种假设（即与预设最吻合的假说）而非其他假设；
- d. 是学习的必要前提，因为归纳学习是一个病态的问题。

6. 机器学习算法如何处理机器学习是一个病态问题的现实？

由于机器学习是错误的，我们必须对我们认为从描述性特征到目标特征的真实映射应该是什么样子**做出一些额外的假设**，以便利用我们拥有的数据找到一个独特的解决方案。我们为学习成为可能而做出的一组假设称为学习算法的**归纳偏差**。

7. 当使用不适当的归纳偏置时会出什么问题？

有两种可能的负面结果：

如果学习算法的归纳偏差限制搜索**仅考虑简单模型**，我们可能会从此搜索中排除任何成功捕获数据中关系的模型。换句话说，真正的模型是算法无法实现的。**在这种情况下，会发生欠拟合。**

如果学习算法的归纳偏差允许搜索**考虑过于复杂的模型**，则该算法可能会锁定训练集中不相关的因素。换句话说，模型**将过度拟合训练数据**。

8. 人们常说，预测数据分析项目 80% 的工作是在 CRISP-DM 的业务理解、数据理解和数据准备阶段完成的，只有 20% 用于建模、评估和部署阶段。你认为为什么会这样？

有一句古老的格言：“**垃圾进，垃圾出**”。无论如何使用机器学习算法或如何调整它们，不可能从坏数据中构建好的模型。

因此，在开始任何建模工作之前，花费足够的时间收集、分析和准备数据至关重要。这项工作主要在 CRISP-DM 流程的**数据理解和数据准备**阶段完成。CRISP-DM 流程的业务理解和数据理解阶段还涉及业务领域专家与预测数据分析和机器学习专家之间的大量沟通，以充分定义将要解决的问题。这种沟通不可避免地很耗时。然而，至关重要的是，交付的最终模型实际上适合业务。

第二章：从数据到洞察到决策

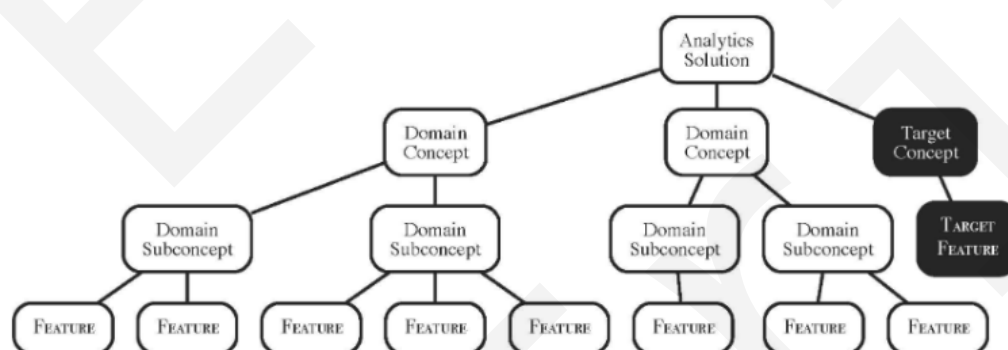
定义分析解决方案是 CRISP-DM 流程的**业务理解**阶段中最重要的任务。

Designing the **Analytics Base Table**, ABT 设计主要位于 CRISP-DM 流程的**数据理解**阶段，但也与**业务理解**和**数据准备**阶段重叠。

特征	Descriptive Features						Target Feature
	---	---	---	---	---	---	---
	---	---	---	---	---	---	---
	---	---	---	---	---	---	---
	---	---	---	---	---	---	---
	---	---	---	---	---	---	---
	---	---	---	---	---	---	---

分析基表的一般结构 - 描述性特征和目标特征

定义特征的一个好方法是确定关键的**领域概念**，然后基于这些概念来构建。领域概念是一种高级抽象，它描述了预测主题的某些特征，我们从中推导出一组将包含在 ABT 中的具体特征。



数据分类：

连续的 continuous：

Numeric: True numeric values that allow arithmetic operations (e.g., price, age)

数值：允许算术运算的真数值（例如，价格、年龄）

Interval: Values that allow ordering and subtraction, but do not allow other arithmetic operations (e.g., date, time)

间隔：允许排序和减法，但不允许其他算术运算（例如日期、时间）的值

分类的 categorical:

Ordinal: Values that allow ordering but do not permit arithmetic (e.g., size measured as small, medium, or large)

序数: 允许排序但不允许算术的值 (例如, 大小测量为小、中或大)

Categorical: A finite set of values that cannot be ordered and allow no arithmetic (e.g., country, product type)

分类: 一组有限的值, 不能排序, 不允许算术 (例如, 国家、产品类型)

Binary: A set of just two values (e.g., gender)

二元的: 仅包含两个值的集合 (例如, 性别)

Textual: Free-form, usually short text data (e.g., name, address)

文本: 自由格式, 通常是简短的文本数据 (例如姓名、地址)

ID	NAME	DATE OF BIRTH	GENDER	CREDIT RATING	COUNTRY	SALARY
0034	Brian	22/05/78	male	aa	ireland	67,000
0175	Mary	04/06/45	female	c	france	65,000
0456	Sinead	29/02/82	female	b	ireland	112,000
0687	Paul	11/11/67	male	a	usa	34,000
0982	Donald	01/12/75	male	b	australia	88,000
1103	Agnes	17/09/76	female	aa	sweden	154,000

ABT 中的特征可以有两种类型：

原始特征

派生特征

有许多常见的派生特征类型：

聚合 Aggregates

例如，值的计数、总和、平均值、最小值或最大值.....

标志 Flags

指示数据集中存在或不存在某些特征的二进制特征。

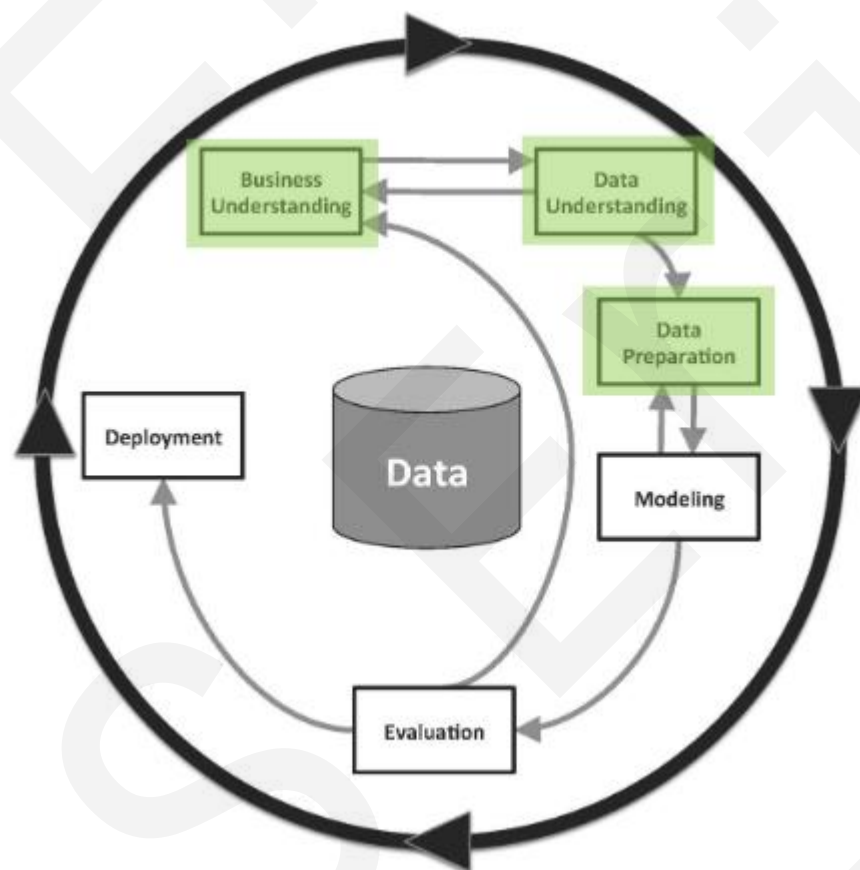
比率 Ratios

比率是捕获两个或多个原始数据值之间关系的连续特征。

映射 Mappings

映射用于将连续要素转换为分类要素

通常用于减少模型必须处理的唯一值的数量。



第三章：数据探索

连续特征

1. 集中趋势 Central Tendency

任何集中趋势的度量都只是一个近似值。

(1) 均值 Mean

$$\bar{a} = \frac{1}{n} \sum_{i=1}^n a_i$$

(2) 中位数 Median

2. 变化 Variation

(1) 范围 Range

$$Range = Max(a) - Min(a)$$

(2) 方差 Variance

$$var(a) = \frac{\sum_{i=1}^n (a_i - \bar{a})^2}{n - 1}$$

注意分母为 n-1，取无偏方差

(3) 标准差 Standard Deviation

$$sd(a) = \sqrt{var(a)} = \sqrt{\frac{\sum_{i=1}^n (a_i - \bar{a})^2}{n - 1}}$$

(4) 百分位数 Percentiles

$$i^{th} percentile = (1 - index_f) \times a_{index_w} + index_f \times a_{index_w+1}$$

其中 index_w 是索引的整个部分，index_f 是小数部分

a[index_w]是有序列表中位置 index_w 的值。

(5) 四分位距 Inter-Quartile Range (IQR)

四分位数间距计算为 25% 和 75% 百分位数之间的差值

25th percentile is also known the lower quartile (or 1st quartile)

25%也称为下四分位数（或 1st 四分位数）

75th percentile is also known upper quartile (or 3rd quartile)

hence the name inter-quartile range

75%也称为上四分位数（或 3rd 四分位数），因此称为四分位数间范围
分类特征

（1）频率计数和比例 Frequency counts and Proportions

频率和比例通常以频率表的形式呈现。

（2）众数/众值 Mode

常用：众数 mode 与第二众数 the second mode

总体和样本

样本是指选择进行分析的总体子集，了解总体和样本之间的差异非常重要。

数据可视化

（1）条形图

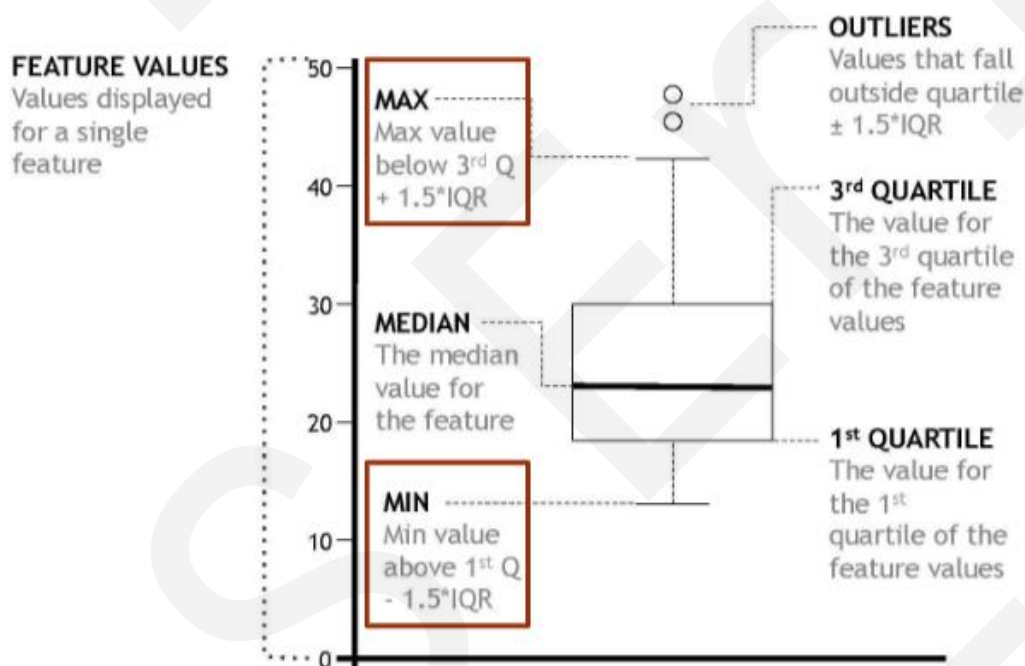
条形图非常适合分类特征，但不适用于连续特征

（2）直方图

通过将变量的范围划分为区间或条柱，我们可以生成直方图

（3）箱形图

箱形图是可视化连续变量的另一种有用方法



数据质量报告

数据质量报告包括表格

使用集中趋势和变化的标准统计度量来描述 ABT 中每个特征的报告的报告。

表格报表附带数据可视化效果：

ABT 中每个**连续特征**的直方图。

ABT 中每个**分类特征**的条形图。

对于**分类特征**，我们应该：

检查众数、第二众数、众数所占 % 和第二众数所占 %，因为它们告诉我们这些特征中最常见的级别，并将确定是否有任何级别在数据集中占主导地位。

对于**连续特征**，我们应该：

检查每个要素的均值和标准差，以了解该要素数据集中值的集中趋势和变化。

检查最小值和最大值，以了解每个要素的可能范围。

均值 $\mu = 0$ 且标准差 $\sigma = 1$ 的正态分布称为标准正态分布。

68 - 95 - 99.7 规则是正态分布的有用特征。

68% 的观测值将在 μ 的 $\pm 1\sigma$ 范围内

95% 的观测值将在 μ 的 $\pm 2\sigma$ 范围内

99.7% 的观测值将在 μ 的 $\pm 3\sigma$ 范围内

数据质量问题被松散地定义为 ABT 中数据的任何异常情况。

(1) 缺失值 missing values

(2) 不规则基数 irregular cardinality

特征的基数与我们预期的不匹配

检查基数列中是否存在基数为 1 的要素

检查基数列中是否有错误标记为连续的分类要素

检查分类特征的基数是否比我们预期的要高得多，给定该特征的定义

检查分类要素的级别数是否非常多，任何超过 50 的级别都值得调查

(3) 异常 Outliers

异常值是远离要素中心趋势的值。

比较中位数、最小值、最大值、第 1 个四分位数和第 3 个四分位数值之间的差距。

如果第 3 个四分位数与最大值之间的差距明显大于中位数和第 3 个四分位数之间的差距，则最大值可能是异常值。

处理数据质量问题

(1) 缺失值 missing values

方法 1：删除任何具有缺失值的要素。

方法 2：应用完整的案例分析。

方法 3：从具有缺失值的要素中导出缺失指标特征。

插补将缺失的特征值替换为基于存在的特征值的合理估计值。

最常见的插补方法是将特征的缺失值替换为该特征的集中趋势的度量值。

我们不愿意对缺失超过其值的 30% 的特征使用插补，并强烈建议不要对缺失超过 50% 值的特征使用插补。

(2) 不规则基数 irregular cardinality 同上

(3) 异常 Outliers

一种常见的方法：将下限阈值设置为第 1 个四分位数值减去四分位数间距的 1.5 倍，将上限阈值设置为第 3 个四分位数加上四分位数间距的 1.5 倍。

另一种常用的方法：使用特征的平均值正负 2 倍标准差。

高级数据探索 Advanced Data Exploration

可视化要素之间的关系

散点图

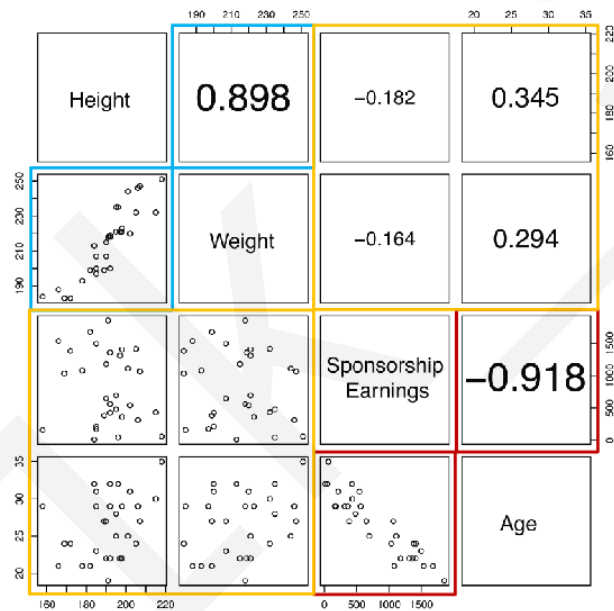
散点图矩阵

条形图

堆叠条形图

连续要素直方图

箱形图



测量协方差和相关性

协方差

$$cov(a, b) = \frac{1}{n-1} \sum_{i=1}^n \left((a_i - \bar{a}) \times (b_i - \bar{b}) \right)$$

注意分母

协方差的取值范围为正负无穷

接近零的值表示要素之间几乎没有关系

相关性是协方差的归一化形式，范围介于 -1 和 +1 之间。

$$corr(a, b) = \frac{cov(a, b)}{sd(a) \times sd(b)}$$

其中 CoV (a, b) 是特征 A 和 B 之间的协方差，

SD (A) 和 SD (B) 分别是 A 和 B 的标准差。

没有相关性的特征被称为独立

对于多个特征：协方差矩阵与相关性矩阵

相关性是衡量两个连续特征之间关系的一个很好的指标，但它绝不是完美的。

图表的重要性：ANScombe的四组数据

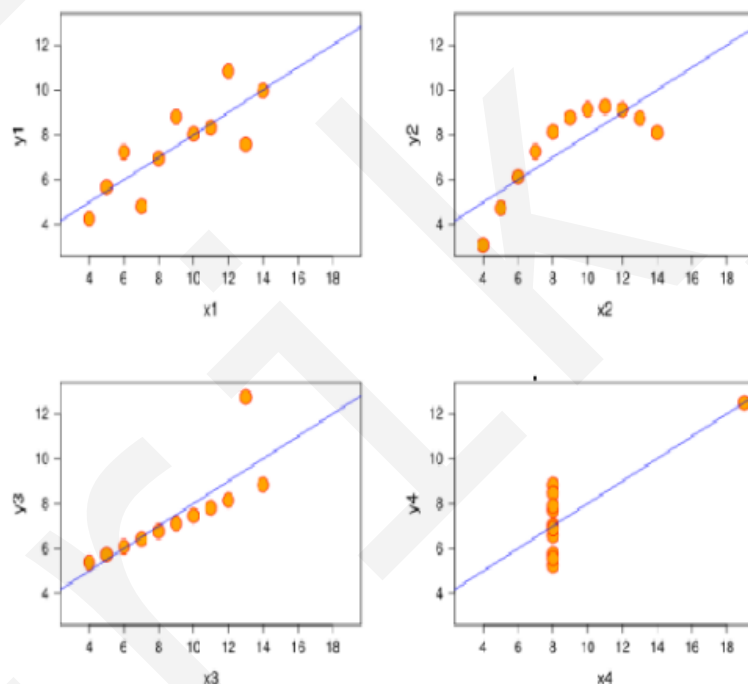


图3.12 安斯科姆的四重奏。对于所有四个样本，相关度量返回相同的值（0.816），即使每种情况下特征之间的关系都非常不同。

关于相关性要记住的最重要的事情是，相关性并不一定意味着因果关系。

数据准备

正则化 Normalization

范围正则化 range normalization

$$a'_i = \frac{a_i - \min(a)}{\max(a) - \min(a)} \times (high - low) + low$$

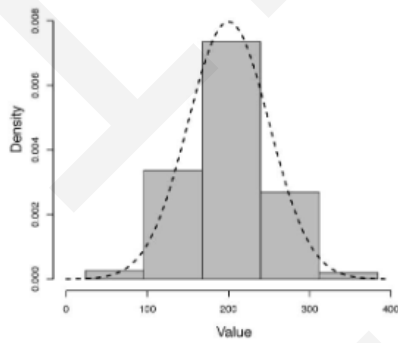
标准化 standardize

$$a'_i = \frac{a_i - \bar{a}}{sd(a)}$$

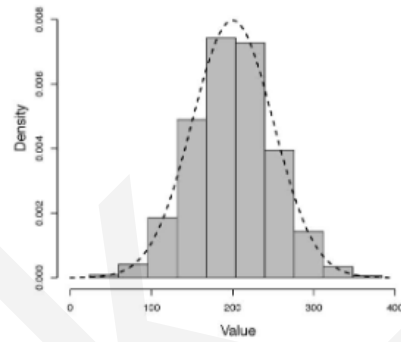
分箱 Binning

分箱涉及将连续要素转换为分类要素。箱子个数需要权衡

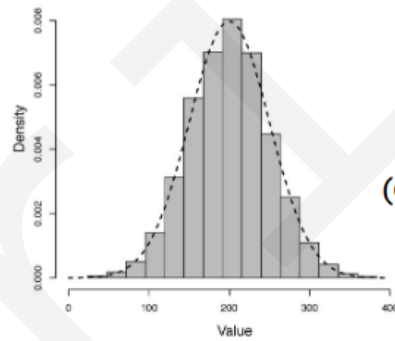
等宽分箱



(a) 5 Equal-width bins

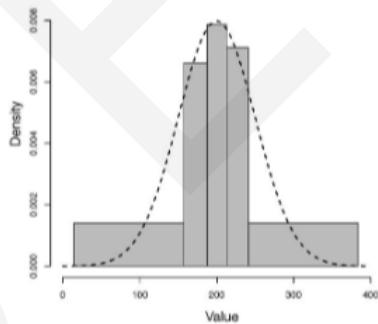


(b) 10 Equal-width bins

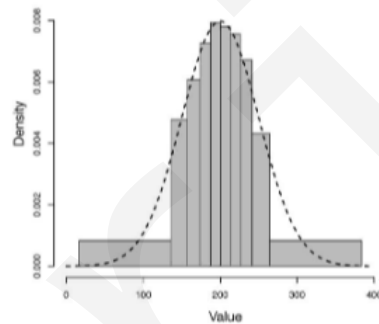


(c) 15 Equal-width bins

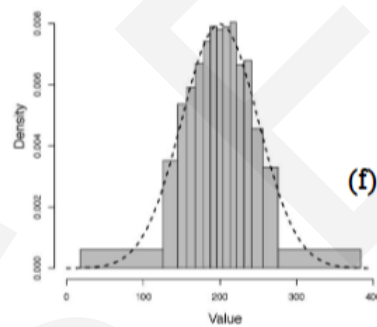
等频分箱



(d) 5 Equal-frequency bins



(e) 10 Equal-frequency bins



(f) 15 Equal-frequency bins

采样 Sampling

我们在抽样时需要小心，以确保生成的数据集仍然代表原始数据，并且在此过程中不会引入意外的偏差。

顶部采样

顶部采样只是从数据集中选择前 $s\%$ 的实例来创建样本。存在引入偏差的严重风险，因为样本将受到原始数据集的任何排序的影响。

随机抽样

我们推荐的默认随机抽样会从大型数据集中随机选择一定比例的 $s\%$ 实例，以创建较小的数据集。

分层抽样

分层抽样是一种抽样方法，可确保在采样数据集中保持特定分层要素水平的相对频率。

数据集中的实例被划分为多个组（或分层），其中每个组仅包含具有特定分层特征级别的实例

随机选择每个层中 $s\%$ 的实例

这些选择组合在一起，给出原始数据集的 $s\%$ 的总体样本。

欠采样/过采样：与分层抽样相比，有时我们希望样本包含特定特征水平与原始数据集中分布的不同相对频率。

欠采样：首先将数据集划分为多个组，其中每个组仅包含具有特定级别的实例，以便对要素进行欠采样。

然后，每个包含比最小实例多的组按适当的百分比随机采样，以创建一个子集，该子集是欠采样的目标大小。

然后将这些欠采样组组合在一起，以创建整体欠采样数据集。

过采样：解决了与欠采样相同的问题，但方式相反。

将数据集划分为组后，最大组中的实例数成为过采样目标大小。

然后，从每个较小的组中，我们使用随机抽样和替换创建一个包含该数量实例的样本。

这些较大的样本组合在一起形成整体过采样数据集。

第四章：基于信息的学习

考虑问题的不同答案的影响，找出哪些特征是提问信息量最大的特征

决策树

树中的每个非叶节点（根节点和内部节点）都指定要对查询的描述性特征之一执行的测试。

每个叶节点都为查询指定一个预测分类。

首选使用较少测试的决策树（较浅的树）

我们需要做的只是一个集合纯度的计算度量：熵

$$\begin{aligned} H(t) &= - \sum_{i=1}^l P(t=i) \times \log_s(P(t=i)) \\ &= \sum_{i=1}^l P(t=i) \times \log_s \frac{1}{P(t=i)} \\ 0 &\leq H(t) \leq \log l \end{aligned}$$

【熵的单位：bits】以 $\log_2$ 为底数计算出来的结果单位为 bit

信息增益

$$\begin{aligned} H(t, D) &= - \sum_{l \in \text{levels}(t)} \left(P(t=l) \times \log_s(P(t=l)) \right) \\ \text{rem}(d, D) &= \sum_{l \in \text{levels}(d)} \frac{|D_{d=l}|}{|D|} \times H(t, D_{d=l}) \\ IG(d, D) &= H(t, D) - \text{rem}(d, D) \end{aligned}$$

原始熵 - 分割后的子集加权熵 = 熵减 = 信息增益

标准方法：ID3 算法

ID3 算法以递归、深度优先的方式构建树，从根节点开始，一直到叶节点。

1. 首先使用信息增益选择要测试的最佳描述性特征。
2. 将根节点添加到树中，并用选定的测试特征进行标记。
3. 使用测试对训练数据集进行分区。每个分区节点中生长出一个分支。
4. 然后，使用训练集的相关分区代替完整的训练集，并从进一步测试中排除所选测试特征，对每个分支重复该过程。

该算法定义了递归停止并构造叶节点的三种情况：

1. 数据集中的所有实例都具有相同的分类（目标特征值），然后返回一个叶节点树，该分类作为其标签。**【达到 100%纯度，返回标签】**
2. 待测试的特征集为空，然后返回一个叶节点树，其中数据集的大多数类作为其分类。**【没有可用特征了，返回众数 target】**
3. 数据集为空，返回一个叶节点树，其中数据集的多数类位于进行递归调用的父节点上。**【没有可分数据了，返回父节点】**

算法进阶：

针对 ID3 算法提出了许多扩展和变体：

1. 使用不同的测量（基尼系数、增益比）
2. 处理连续描述性特征
3. 处理连续目标
4. 修剪以防止过度拟合
5. 使用决策树作为集成的一部分（Random Forests）

1. 使用不同的测量（基尼系数、增益比）

信息增益比 Information gain ratio

$$GR(d, D) = \frac{IG(d, D)}{-\sum_{l \in \text{levels}(d)} P(d = l) \times \log_2 P(d = l)}$$

信息增益/原始熵【归一化】

信息增益的优点是计算成本低于信息增益比。

如果数据描述性特征域中的值数量存在差异，则信息增益率可能是更好的选择。

描述性特征选择指标的有效性可能因域而异。

基尼指数 Gini index:

$$Gini(t, D) = 1 - \sum_{l \in \text{levels}(t)} P(t = l)^2$$

信息增益可以通过用基尼指数替换熵度量来使用基尼指数计算。

基尼指数分数始终在 0 和 1【无穷标签分类的极限】之间，这可能使跨特征比较基尼指数变得更加容易。

基尼指数可以被认为是计算如果根据数据集中分类的分布对数据集中的实例进行分类，则对数据集中的实例进行错误分类的频率。

当数据集中的所有实例都具有相同的目标标签时，基尼指数= 0，

当有 k 个可能的目标水平具有相等的可能性时，基尼指数=1-1/k

对于植被数据集，使用基于基尼指数的信息增益生成的决策树将与使用基于熵的信息增益生成的决策树相同。

2. 处理连续描述性特征

处理连续值描述性特征的最简单方法是通过定义阈值并使用此阈值根据连续描述性特征的值对实例进行分区，将其转换为布尔特征。

数据集中的实例根据连续特征值进行排序。

然后，选择排序中具有不同分类的相邻实例作为可能的阈值点。

通过计算每个分类转移边界的信息增益并选择信息增益最高的边界作为阈值，可以找到最佳阈值。

设置阈值后，动态创建的新布尔特征可以与其他分类特征竞争，以选择作为该节点的分割特征。

随着树的生长，可以在每个节点上重复此过程。

Table 4.9: Dataset for predicting the vegetation in an area sorted by the continuous ELEVATION feature.

ID	STREAM	SLOPE	ELEVATION	VEGETATION
2	true	moderate	300	riparian
4	false	steep	1 200	chapparal
3	true	steep	1 500	riparian
7	true	steep	3 000	chapparal
1	false	steep	3 900	chapparal
5	false	flat	4 450	conifer
6	true	steep	5 000	conifer

the boundary between d2 and d4 is $\frac{300+1200}{2}=750$

the boundary between d4 and d3 is $\frac{1200+1500}{2}=1350$

the boundary between d3 and d7 is $\frac{1500+3000}{2}=2250$

the boundary between d1 and d5 is $\frac{3900+4450}{2}=4175$

与分类特征不同，连续特征可以在决策树中沿路径的多个点使用，尽管在每次测试中应用于【特征的阈值会有所不同。】

这很重要，因为它允许在路径上考虑连续要素范围内的多个分割。

因此，当我们构建树的其余部分时，我们可以重用特征。

3. 预测连续目标

构建回归树是为了减少树中每个叶节点的训练样本集的方差
为此，我们可以通过在选择最佳属性时调整 ID3 算法以使用**方差度量**而不是分类杂质（熵）度量来做到这一点。

$$d[best] = \arg \min_{d \in d} \sum_{l \in \text{level}(d)} \frac{|D_{d=l}|}{|D|} \times \text{var}(t, D_{d=l})$$

【算法修改】

如果分区中的训练实例数小于某个阈值（如 5%），则停止对数据集进行分区
创建一个叶节点，返回数据分区中目标特征的平均值
信息增益公式替换为方差公式

4. 嘈杂的数据、过拟合和树木修剪

随着树的深入，过度拟合的可能性会增加，因为生成的分类基于越来越小的子集，因为在路径中的每个特征测试后对数据集进行分区。

修剪将导致创建的决策树与用于构建它们的训练集不一致
为了泛化能力而牺牲一致性是很常见的。

两种类型：Pre-pruning, Post-pruning

预修剪：尽早停止递归分区

预修剪也称为前向修剪，计算效率高，适用于小型数据集。

a. 提早停止

策略：

分区中的实例数低于阈值

在节点上测量的信息增益被认为不足以使数据分区有价值

树的深度超出了预定义的限制

b. 剪枝（卡方）：使用统计显著性检验来确定子树的重要性

但是，使用预修剪的归纳算法可能无法创建最有效的树，因为它们会**错过子树中出现的特征之间的交互**，这些交互在考虑父节点时并不明显。

后修剪：允许算法随心所欲地种植树，然后修剪导致过度拟合的树枝。

后修剪依赖于一个条件，该条件可以**区分**对数据的相关方面进行建模的子树和对数据中不相关的随机模式进行建模的子树。**【区分了树到底有没有用】**

建议使用标准来比较决策树在包含给定子树和修剪子树时所做的预测中的错误率。

一些训练数据被留作验证数据集。在训练期间不使用。

使用验证集评估完全生长的树和修剪的树副本所达到的预测准确性。

如果修剪的树副本的性能不比完全生长的树差，则节点是修剪的候选项。

【先随便长，长完再修剪】

它使树木变小，从而使它们更容易解释。

当训练数据中存在噪声（噪声阻尼）时，提高了泛化精度。

减少错误修剪（Quinlan, 1987）是基于错误率的后期修剪的流行版本。

构建决策树直至完成，然后以迭代、自下而上、从左到右的方式搜索该树，以查找可以修剪的子树。

将在每个子树的根节点上对验证数据集中的实例进行的预测所产生的错误率与在子树的叶子上进行的预测所产生的错误率进行比较。

如果子树根节点处的错误率小于或等于叶子处的组合错误率，则修剪子树。

【子树没有带来增益】

【训练集建树，验证集剪树】

5. 集成学习

由一组模型组成的预测模型称为模型集合。

为了使此方法起作用，**集成中的模型必须彼此不同**。

集成模型的两个定义特征：

1. 通过使用数据集的修改版本诱导每个模型，从同一数据集构建多个不同的模型。
2. 通过聚合融合中不同模型的预测来进行预测。

多个弱模型组合成强模型

两种创建集成的标准方法：

1. 装袋 Bagging

当我们使用 bagging（或 bootstrap 聚合）时，集成中的每个模型都是在数据集的**随机样本**上训练的，称为 bootstrap 样本。

每个随机样本的大小与数据集相同，并使用替换采样。

因此，每个引导程序示例都将缺少数据集中的一些实例，因此每个引导程序示例都将不同，这意味着在不同引导程序样本上训练的模型也将不同。

决策树归纳算法特别适合与装袋一起使用，因为它们对数据集中的变化非常敏感。

当 bagging 与决策树一起使用时，每个引导样本仅使用数据集中随机选择的描述性特征子集。称为子空间采样。

装袋、子空间采样和决策树的组合称为随机森林模型。

2. 提高 Boosting

通过迭代创建模型并将它们添加到整体中来提升工作。

添加预定义数量的模型后，迭代将停止。

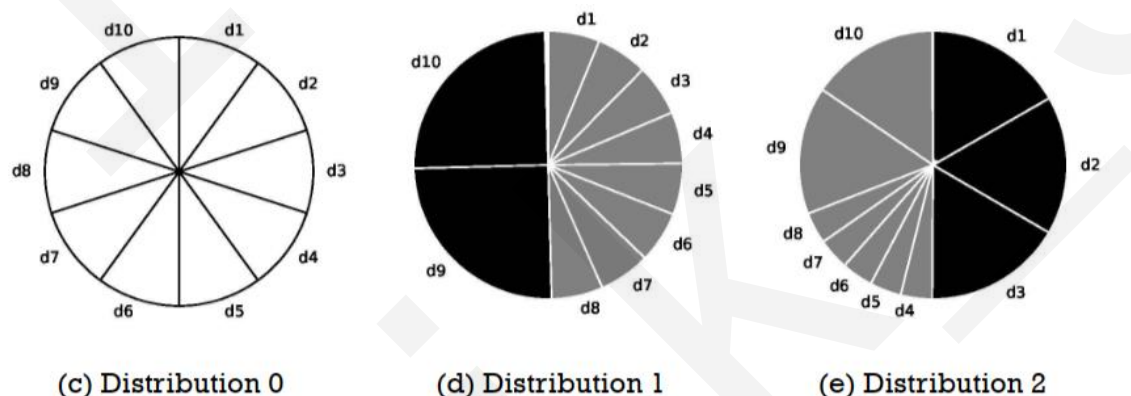
当我们使用提升时，添加到融合中的每个新模型都有偏差，以更多地关注以前模型错误分类的实例。

将每个模型添加到融合后，在训练数据上对其进行测试，并且模型获得正确的实例的权重将减少，并且模型不正确的实例的权重将增加。

在每次训练迭代期间，算法：

1. 诱导模型并计算总误差，方法是对模型所做的预测不正确的训练实例的权重求和。
2. 使用以下方法增加错误分类的实例的权重： $\mathbf{w}[i] \leftarrow \mathbf{w}[i] \times \left(\frac{1}{2 \times \varepsilon}\right)$
3. 降低正确分类的实例的权重： $\mathbf{w}[i] \leftarrow \mathbf{w}[i] \times \left(\frac{1}{2 \times (1 - \varepsilon)}\right)$
4. 为模型计算置信因子 α ，使 α 随着 ε 的减小而增加： $\alpha = \frac{1}{2} \times \log_e \left(\frac{1 - \varepsilon}{\varepsilon}\right)$

【正确错误各分一半权重因子，正确样例多所以权重变小，错误样例少所以权重变大，更新完后总权重仍然为 1， ε 表示错误样例所占总权重】



与提升相比，bagging 更容易实现和并行化，因此在易用性和训练时间方面可能更好。

实证结果表明：

提升决策树集成是针对包含多达 4,000 个描述性特征的数据集测试的模型中性能最佳的模型。

随机森林集合（基于 bagging）对于包含超过 4,000 个要素的数据集表现更好。

提升系统容易出现过度拟合，在具有大量特征的域中，过度拟合成为一个严重的问题。

优势

能够对描述性特征之间的相互作用进行建模

【决策树相对而言，对维度的诅咒很健壮】

【相对而言，如果使用修剪，则对数据集中的噪声具有鲁棒性】

劣势

【在处理连续要素时，树会变大】

如果有很多特征，它们可能会过度拟合数据（维度的诅咒）

不适合对随时间变化的概念（概念漂移）进行建模，因为它们需要重新训练

梯度提升 Gradient Boosting

$$M_0(d) = \frac{1}{n} \sum_{i=1}^n t_i$$

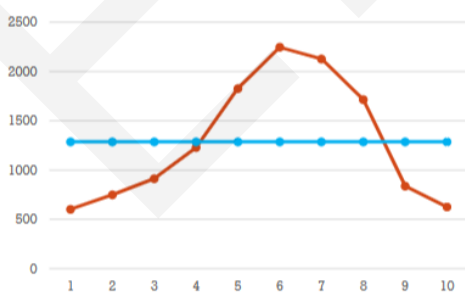
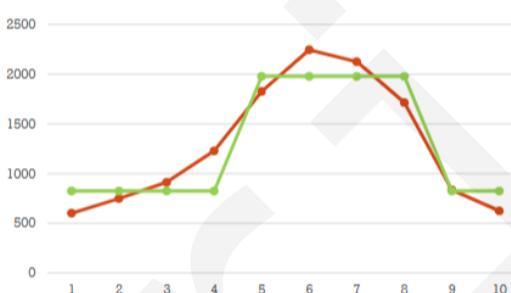
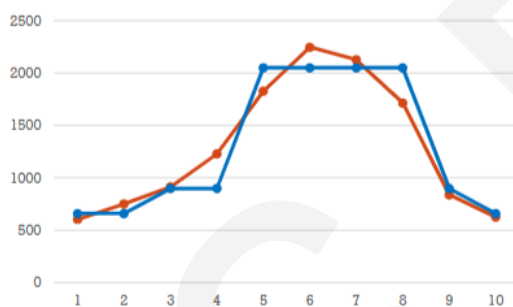
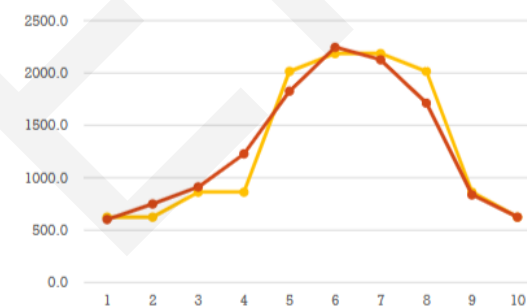
$$M_1(d) = M_0(d) + M_{\Delta 1}(d)$$

$$M_i(d) = M_{i-1}(d) + M_{\Delta i}(d)$$

ID	TEMP	TENT	M0	t-M0	MD1	M1	t-M1	MD2	M2	t-M2	MD3	M3
1	4	602	1287.1	-685.1	-460.9	826.2	-224.2	-167.2	659.0	-57.0	-34.1	624.9
2	5	750	1287.1	-537.1	-460.9	826.2	-76.2	-167.2	659.0	91.0	-34.1	624.9
3	7	913	1287.1	-374.1	-460.9	826.2	86.8	71.6	897.8	15.2	-34.1	863.7
4	12	1229	1287.1	-58.1	-460.9	826.2	402.8	71.6	897.8	331.2	-34.1	863.7
5	18	1827	1287.1	539.9	691.4	1978.5	-151.5	71.6	2050.1	-223.1	-34.1	2016.0
6	23	2246	1287.1	958.9	691.4	1978.5	267.5	71.6	2050.1	195.9	136.4	2186.5
7	27	2127	1287.1	839.9	691.4	1978.5	148.5	71.6	2050.1	76.9	136.4	2186.5
8	28	1714	1287.1	426.9	691.4	1978.5	-264.5	71.6	2050.1	-336.1	-34.1	2016.0
9	32	838	1287.1	-449.1	-460.9	826.2	11.8	71.6	897.8	-59.8	-34.1	863.7
10	35	625	1287.1	-662.1	-460.9	826.2	-201.2	-167.2	659.0	-34.0	-34.1	624.9

TENT: 目标值【每一次迭代只取连续段，跨越标签也要取连续段】

M0: 目标值的均值, t-M0: 计算均值相对于目标值的偏差, MD1: 分组均值

(a) M_0 (b) M_1 (c) M_2 (d) M_3

第五章：基于相似性的学习

基于相似性的机器学习方法源于这样一种想法，即做出预测的最佳方法是简单地查看过去运作良好的方法，然后再次预测相同的事物。

如果你试图对某件事进行分类，那么你应该搜索你的记忆以找到相似的东西，并用与记忆中最相似的东西相同的类来标记它

基于相似性学习的基本原理是：

特征空间

特征空间是一个抽象的 m 维空间，它是通过将 ABT 中的每个描述性特征作为参考空间的轴而创建的

相似性指标

相似度指标根据特征空间衡量两个实例之间的相似性

从数学上讲，指标必须符合以下四个条件：

- 非负性： $Metric(a, b) \geq 0$
- 唯一性： $Metric(a, b) = 0 \leftrightarrow a = b$
- 对称性： $Metric(a, b) = Metric(b, a)$
- 三角不等式： $Metric(a, b) \leq Metric(a, c) + Metric(b, c)$

其中 $metric(a, b)$ 是一个函数，它返回两个实例 a 和 b 之间的距离。

欧几里得距离 Euclidean distance

$$Euclidean(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^m (\mathbf{a}[i] - \mathbf{b}[i])^2}$$

曼哈顿距离 Manhattan distance

$$Manhattan(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^m abs(\mathbf{a}[i] - \mathbf{b}[i])$$

闵可夫斯基距离 Minkowski distance

$$Minkowski(\mathbf{a}, \mathbf{b}) = \left[\sum_{i=1}^m abs(\mathbf{a}[i] - \mathbf{b}[i])^p \right]^{\frac{1}{p}}$$

$p = 1$ 的闵可夫斯基距离是曼哈顿距离， $p = 2$ 是欧几里得距离

p 的值越大，越强调值差异较大的特征，因为这些差异被提高到 p 的幂

标准方法：KNN 算法

1. 遍历内存中的实例，并找到与特征空间中的查询位置距离最短的实例。
2. 对查询进行预测，该预测等于最近邻的目标要素的值。

最近邻算法的一大优点是，我们可以添加新数据来非常轻松地更新模型。

监督式机器学习的两个重要且相关的方面：

监督式机器学习基于平稳性假设，即数据不会随时间变化 - **保持静止**。

在分类的上下文中，监督式机器学习会创建模型，用于区分它们所来自的**数据集中存在的类**。

因此，如果对分类模型进行训练以区分狮子、青蛙和鸭子，则该模型会将查询分类为狮子、青蛙或鸭子；**即使查询实际上是鸭嘴兽。【无法识别新类】**

算法进阶：

1. 处理嘈杂的数据
2. 数据规范化
3. 预测连续目标
4. 其他相似性衡量标准
5. 特征选择
6. 高效的内存搜索

1. 处理嘈杂的数据

该算法对噪声很敏感，因为训练数据的描述或标记中的任何错误都会导致错误的局部模型，从而导致不正确的预测。

减轻数据集中噪声对最近邻算法的影响的最直接方法是稀释算法对单个（可能是噪声）实例的依赖性。

$$\mathbf{M}_k(\mathbf{q}) = \arg \max_{l \in \text{level}(t)} \sum_{i=1}^k \delta(t_i, l)$$

k 设置得太低，则存在算法对数据中的噪声敏感和过度拟合的风险。

将 k 设置得太高，我们将面临丢失数据真实模式和欠拟合的风险。

当我们处理不平衡的数据集时，与将 k 设置为高值相关的风险尤为严重。

不平衡数据集是指一个目标级别包含的实例明显多于另一个目标级别的实例的数据集。

在这些情况下，随着 k 的增加，大多数目标水平开始主导特征空间。

解决此问题的最常见方法是执行评估实验，以研究具有不同 k 值的模型的性能，并选择性能最佳的模型。

解决如何设置 k 问题的另一种方法

是使用加权 k 最近邻方法。

$$\mathbf{M}_k(\mathbf{q}) = \arg \max_{l \in \text{level}(t)} \sum_{i=1}^k \frac{1}{\text{dist}(\mathbf{q}, \mathbf{d}_i)^2} \times \delta(t_i, l)$$

在数据集中处理噪声没有灵丹妙药，要接受不完美。

2. 数据规范化

采用不同值范围的要素引起的预测出错，这相当于具有不同方差的特征同时使用，造成一部分特征抢占主导。

我们可以使用规范化来调整这一点；范围归一化的公式为：

$$a'_i = \frac{a_i - \min(a)}{\max(a) - \min(a)} \times (\text{high} - \text{low}) + \text{low}$$

对于几乎所有的机器学习算法来说，对数据进行归一化是一件重要的事情，而不仅仅是 KNN！

3. 预测连续目标

采用 k 最近邻方法处理连续目标特征相对容易。

返回邻域中的平均值：

$$\mathbf{M}_k(\mathbf{q}) = \frac{1}{k} \sum_{i=1}^k t_i \quad \mathbf{M}_k(\mathbf{q}) = \frac{\sum_{i=1}^k \frac{1}{\text{dist}(\mathbf{q}, \mathbf{d}_i)^2} \times t_i}{\sum_{i=1}^k \frac{1}{\text{dist}(\mathbf{q}, \mathbf{d}_i)^2}}$$

通常，当特征空间填充良好时，标准 k 最近邻模型和加权 k 最近邻模型将产生非常相似的结果。

但对于仅稀疏填充特征空间的数据集，加权 knn 通常会预测更准确。

4. 其他相似性衡量标准

我们几乎可以互换使用术语相似性和距离，因为我们经常根据特征空间中两个实例之间的距离来判断它们之间的相似性。

区分指标和索引：

度量必须满足四个标准：非负性、恒等性、对称性和三角不等式。

不满足所有这四个标准的相似性度量称为索引。

二元描述性特征的相似性索引 Indexes

如果数据集中的描述性特征是二进制的，则通常最好使用相似性索引来定义实例之间的相似性，特别是根据特征的共存或共不存在来定义实例之间的相似性，而不是基于距离的索引。

共存 (CP) co-presence

共缺 (CA) co-absence

存在-不存在 (PA) presence-absence 【query 有】

不存在-存在 (AP) absence-presence 【query 没有】

(1) Russel-Rao

$$sim_{RR}(\mathbf{q}, \mathbf{d}) = \frac{CP(\mathbf{q}, \mathbf{d})}{|\mathbf{q}|}$$

(2) Sokal-Michener

$$sim_{SM}(\mathbf{q}, \mathbf{d}) = \frac{CP(\mathbf{q}, \mathbf{d}) + CA(\mathbf{q}, \mathbf{d})}{|\mathbf{q}|}$$

(3) Jaccard

$$sim_J(\mathbf{q}, \mathbf{d}) = \frac{CP(\mathbf{q}, \mathbf{d})}{CP(\mathbf{q}, \mathbf{d}) + PA(\mathbf{q}, \mathbf{d}) + AP(\mathbf{q}, \mathbf{d})}$$

$|\mathbf{q}|$ 表示 query 的特征数

除了强调 Jaccard 索引对稀疏二进制数据有用之外，我们不能给出如何在这些索引之间进行选择的硬性规定。

余弦相似度 (index)

$$\begin{aligned} \text{sim}_{\cosINE}(\mathbf{a}, \mathbf{b}) &= \frac{\mathbf{a} \cdot \mathbf{b}}{\sqrt{\sum_{i=1}^m \mathbf{a}[i]^2} \times \sqrt{\sum_{i=1}^m \mathbf{b}[i]^2}} \\ &= \frac{\sum_{i=1}^m (\mathbf{a}[i] \times \mathbf{b}[i])}{\sqrt{\sum_{i=1}^m \mathbf{a}[i]^2} \times \sqrt{\sum_{i=1}^m \mathbf{b}[i]^2}} \end{aligned}$$

实例之间的余弦相似度将在 $[0, 1]$ 范围内:

1 表示最大相似度

0 表示最大差异。

当描述数据集中实例的描述性特征彼此相关时, 余弦相似性是一种特别有用的相似性度量。

这种归一化使余弦相似性在我们对值在一组描述性特征中的相对分布而不是值本身的大小感兴趣的场景中如此有用。

余弦相似性也是具有非二元特征的稀疏数据 (即具有大量零值的数据集) 的适当相似性指数, 因为点积在其计算中基本上会忽略共同缺失 ($0 \times 0 = 0$)。

马氏距离 (公制)

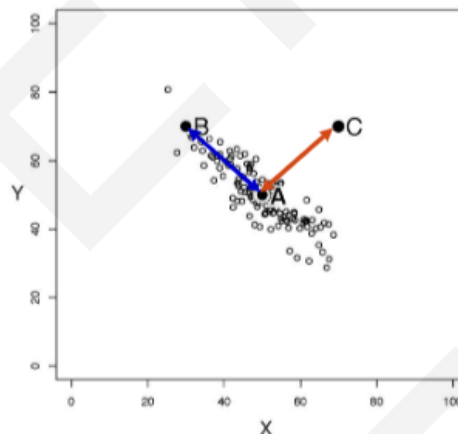
马氏距离与我们研究过的其他距离指标不同, 因为它允许我们在判断相似性时考虑数据集中实例的分布程度。

mahalanobis 距离使用协方差来缩放距离:

沿数据集分布大量方向的距离按比例缩小

沿数据集紧密排列的方向的距离按比例放大

In Figure 5.15(b) the Mahalanobis distance between **B** and **A** will be **less**, whereas in Figure 5.15(c) the opposite will be true.



$$Mahalanobis(\mathbf{a}, \mathbf{b}) = \sqrt{[\mathbf{a}[1] - \mathbf{b}[1], \dots, \mathbf{a}[m] - \mathbf{b}[m]] \times \left(\sum\right)^{-1} \times \begin{bmatrix} \mathbf{a}[1] - \mathbf{b}[1] \\ \dots \\ \mathbf{a}[m] - \mathbf{b}[m] \end{bmatrix}}$$

The term Σ^{-1} represents the **inverse covariance matrix** computed across **all instances** in the dataset.

$$\Sigma \times \Sigma^{-1} = \mathbf{I}$$

$$\Sigma_{\{a,b,\dots,z\}} = \begin{bmatrix} \text{var}(a) & \text{cov}(a,b) & \dots & \text{cov}(a,z) \\ \text{cov}(b,a) & \text{var}(b) & \dots & \text{cov}(b,z) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(z,a) & \text{cov}(z,b) & \dots & \text{var}(z) \end{bmatrix}$$

$$\text{var}(a) = \frac{\sum_{i=1}^n (a_i - \bar{a})^2}{n-1}$$

$$\text{cov}(a,b) = \frac{1}{n-1} \sum_{i=1}^n \left((a_i - \bar{a}) \times (b_i - \bar{b}) \right)$$

与欧几里得距离类似，马哈拉诺比斯距离平方了特征的差异。

但它也会重新缩放差异（使用逆协方差矩阵），以便所有特征都具有单位方差并消除协方差的影响

5. 特征选择

直观地说，向数据集添加更多描述性特征可以提供有关每个实例的更多信息，并且应该会产生更准确的预测模型。

然而，随着数据集中描述性特征数量的增加，经常会出现一个点，即继续向数据集添加新特征会导致诱导模型的预测能力下降。

为了随着描述性特征数量的增加而保持特征空间的采样密度，我们需要指数级地增加实例数量。

描述性特征的数量和特征空间中实例密度之间的这种权衡被称为维度诅咒。

避免维度诅咒的最佳方法是将数据集中描述性特征的数量限制为尽可能小的集合：特征选择

最流行和最直接的特征选择方法是排名和修剪。

排名和修剪方法的缺点是，每个特征的预测性都是独立于数据集中的其他特征进行评估的。这会导致不理想的结果，即排名和修剪可以排除交互特征并包含冗余特征。

特征选择作为**贪婪的局部搜索**问题，其中，搜索空间中的每个状态指定可能要素的子集。

当被描述为贪婪的局部搜索问题时，特征选择是根据由以下阶段组成的迭代过程来定义的：

1. 子集生成

生成一组候选特征子集，这些子集是当前最佳特征子集的后继者。

2. 子集选择

从子集生成组件生成的候选特征子集中选择搜索过程最需要移动到的特征子集。

3. 终止条件

确定搜索过程应何时停止。

前向顺序选择（FSS）

搜索以无要素的状态开始。

当前最佳要素子集的后继者是一组要素子集，只需添加一个额外要素即可从当前最佳子集生成这些要素子集。

当没有可访问要素子集优于当前子集时，它将终止。

向后顺序选择（BSS）

它从一个要素子集开始，包括数据集中所有可能的特征。

当前最佳要素子集的后继者是一组要素子集，只需移除一个额外要素即可从当前最佳子集生成这些要素子集。

当没有可访问要素子集优于或与当前子集一样好时，它将终止。

FSS 和 BSS 都不会考虑添加或删除要素组合的效果, 因此, 不能保证它们能找到要素的绝对最佳子集。

通常, 由于平均而言, FSS 生成的特征子集较小, 因此总体计算成本较低, 但是可能排除交互特征。

BSS 的优点是它允许包含单独可能无法预测的交互特征集, 并且评估较大特征子集的额外计算成本。

如果模型性能比计算考虑更重要, BSS 可能是更好的选择; 否则使用 FSS。

6. 高效的内存搜索【kd Tree】

最近邻算法将整个训练数据集存储在内存中这一事实对算法的时间复杂度产生了负面影响, 尤其是当我们使用大型数据集时。

可以通过投资一些一次性计算来创建实例索引, 从而可以有效地检索最近邻, 从而加快最近邻模型的预测速度。

k-d 树是一种平衡的二叉树, 其中树中的每个节点都索引训练数据集中的—个实例。

要构造 k-d 树:

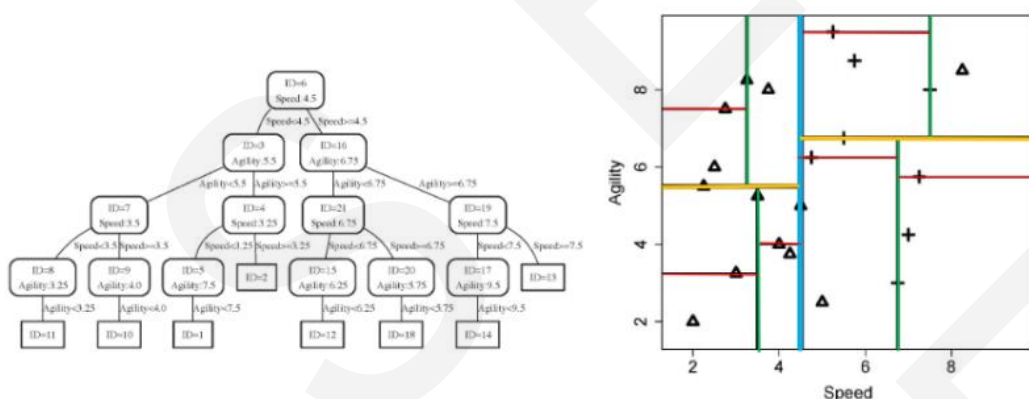
首先选取一个要素, 然后使用要素的中值将数据拆分为两个分区。

然后递归拆分两个新分区中的每一个, ...

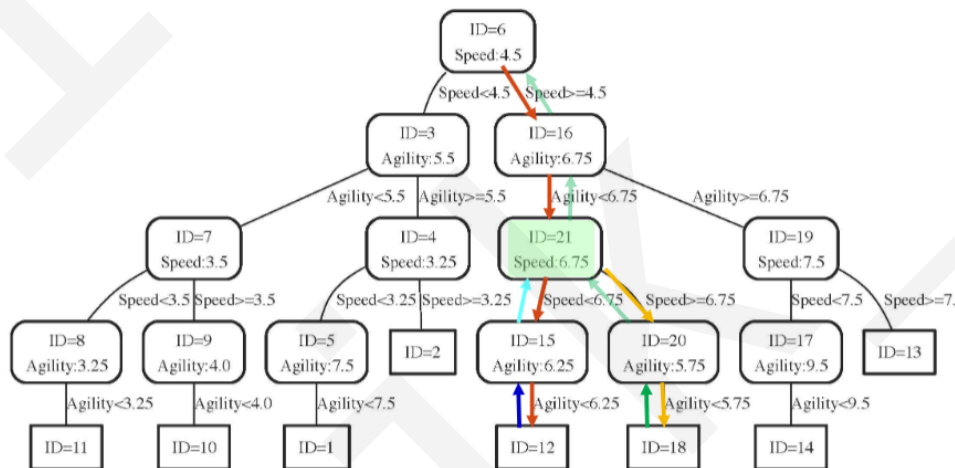
当分区中的实例少于两个时停止递归。

例

完成树形构建过程后



▪ Finding neighbors for query **SPEED=6, AGILITY=3.5**



Require: query instance q and a k -d tree $kdtree$

```

1: best = null
2: best-distance =  $\infty$ 
3: node = descendTree(kdtree,q)
4: while node != NULL do
5:   if distance(q,node) < best-distance then
6:     best = node
7:     best-distance = distance(q,node)
8:   end if
9:   if boundaryDist(q, node) < best-distance then
10:    node = descendtree(node,q)
11:   else
12:    node = parent(node)
13:   end if
14: end while
15: return best
  
```

预备：定义根节点 null

初始化最佳距离，将 query 下沉到 kd 树叶节点

每个节点要做的三件事：

1. 检查节点是否不为 NULL 【表示返回父节点】
若是则返回存储在 best 中的实例
2. 检查实例与当前节点距离，若更优则更新
更新最佳和最佳距离。
3. 选择它下一步应该移动到哪个节点：

节点的父节点或节点另一个分支下的子树中的节点。

若决策边界到 q 的距离更短，表示边界另一侧也可能存在更优解

【注意 descendtree (node, q) 函数是直接下沉到叶节点，而非向下沉一层】

若决策边界到 q 的距离超过当前最优，表示边界另一侧不可能存在更优解，返回父节点【自叶向根向上遍历】

使用 k -d 树可以节省我们计算查询节点与数据集中所有实例之间的距离。这是使用 k -d 树的好处，当数据集非常大时，这一点尤其明显。

但是，使用 k -d 树并不总是合适的；当实例数多于特征数时， K -D 树效率相当高。作为一个粗略的经验法则，我们应该有大约 2 个描述性特征的实例。一旦这个比率下降， k -d 树的效率就会降低。

第六章：基于概率的学习

基于概率的预测方法在很大程度上基于贝叶斯定理。

我们可以使用可能性的估计来确定应该做出的最可能的预测。更重要的是，我们会根据收集的数据以及每当有额外证据可用时修改这些预测。

概念：

概率函数、联合概率、条件概率、概率分布、联合概率分布……

贝叶斯定理 Bayes Theorem:

给定一组证据，一个事件发生的概率等于证据由事件引起的概率乘以事件本身的概率。

$$P(X|Y) = \frac{P(Y|X)P(X)}{P(Y)}$$

除数是证据的先验概率，此除法用作归一化常数。

这很有用，因为从证据到事件的推理（反向推理）通常比从事件推理到它导致的证据（正向推理）要困难得多。

贝叶斯定理允许我们轻松地在这两种推理之间来回切换。

$$P(\mathbf{q}[1], \dots, \mathbf{q}[m] | t = l) = P(\mathbf{q}[1] | t = l) \times P(\mathbf{q}[2] | \mathbf{q}[1], t = l) \times \dots \times P(\mathbf{q}[m] | \mathbf{q}[m-1], \dots, \mathbf{q}[2], \mathbf{q}[1], t = l)$$

考虑：

确保对于任何条件概率，训练数据集中有足够的实例与条件匹配

若出现原始数据样本中未出现的事件组合，如何计算概率？

独立：

事件之间的完全独立性是相当罕见的。

一个更常见的现象是，如果我们知道第三个事件已经发生，那么两个或更多事件可能是独立的。

这称为**有条件独立**。对于两个事件 X 和 Y ，在给定第三个事件 Z 的知识的情况下是有条件独立的

$$P(X|Y,Z) = P(X|Z)$$

$$P(X,Y|Z) = P(X|Z) \times P(Y|Z)$$

$$P(X|Y) = \frac{P(X,Y)}{P(Y)}$$

$$P(X,Y) = P(X|Y) \times P(Y)$$

$$= P(Y|X) \times P(X)$$

$$P(X|Y) = P(X)$$

$$P(X,Y) = P(X) \times P(Y)$$

如果事件 $t = 1$ 导致事件 $q[1], \dots, q[m]$ 发生, 则事件 $q[1], \dots, q[m]$ 在知道 $t = 1$ 的情况下有条件地相互独立, 链式规则定义可以简化如下:

$$P(\mathbf{q}[1], \dots, \mathbf{q}[m]|t = l)$$

$$= P(\mathbf{q}[1]|t = l) \times P(\mathbf{q}[2]|t = l) \times \dots \times P(\mathbf{q}[m]|t = l)$$

$$= \prod_{i=1}^m P(\mathbf{q}[i]|t = l)$$

利用这一点, 我们可以简化贝叶斯定理中的计算, 假设给定目标特征 1, 描述性特征之间存在条件独立性:

$$P(t = l|\mathbf{q}[1], \dots, \mathbf{q}[m]) = \frac{[\prod_{i=1}^m P(\mathbf{q}[i]|t = l)] \times P(t = l)}{P(\mathbf{q}[1], \dots, \mathbf{q}[m])}$$

没有条件独立性:

$$P(X,Y,Z,W) = P(X|W) \times P(Y|X,W) \times P(Z|Y,X,W) \times P(W)$$

有条件独立性

$$P(X,Y,Z,W) = \underbrace{P(X|W)}_{Factor1} \times \underbrace{P(Y|W)}_{Factor2} \times \underbrace{P(Z|W)}_{Factor3} \times \underbrace{P(W)}_{Factor4}$$

称之为 Factorization 因式分解, 大大简化联合概率计算

分解前计算联合概率分布

$$\mathbf{P}(H,F,V,M) = \begin{bmatrix} P(h,f,v,m), & P(\neg h,f,v,m) \\ P(h,f,v,\neg m), & P(\neg h,f,v,\neg m) \\ P(h,f,\neg v,m), & P(\neg h,f,\neg v,m) \\ P(h,f,\neg v,\neg m), & P(\neg h,f,\neg v,\neg m) \\ P(h,\neg f,v,m), & P(\neg h,\neg f,v,m) \\ P(h,\neg f,v,\neg m), & P(\neg h,\neg f,v,\neg m) \\ P(h,\neg f,\neg v,m), & P(\neg h,\neg f,\neg v,m) \\ P(h,\neg f,\neg v,\neg m), & P(\neg h,\neg f,\neg v,\neg m) \end{bmatrix}$$

分解后计算联合概率分布

$$Factor_1 = \langle P(M) \rangle$$

$$Factor_2 = \langle P(h|m), P(h|\neg m) \rangle$$

$$Factor_3 = \langle P(f|m), P(f|\neg m) \rangle$$

$$Factor_4 = \langle P(v|m), P(v|\neg m) \rangle$$

$$P(H, F, V, M) = P(M) \times P(H|M) \times P(F|M) \times P(V|M)$$

标准方法：朴素贝叶斯

朴素贝叶斯模型返回一个 MAP 预测，其中目标特征级别的后验概率是在给定目标特征级别的实例中描述性特征之间的条件独立性假设下计算的。

$$\mathbf{M}(\mathbf{q}) = \arg \max_{l \in \text{level}(t)} \left(\prod_{i=1}^m P(\mathbf{q}[i] | t = l) \right) \times P(t = l)$$

有条件独立性的假设

在证据中的特征之间，给定目标特征的水平也使得朴素的贝叶斯模型对数据碎片化和维度诅咒相对鲁棒。

朴素贝叶斯的训练很简单！

1. 计算每个目标水平的先验

2. 计算给定每个目标水平的每个特征的条件概率。

ID	CREDIT HISTORY	GUARANTOR/ COAPPLICANT	ACCOMODATION	FRAUD
1	current	none	own	true
2	paid	none	own	false
3	paid	none	own	false
4	paid	guarantor	rent	true
5	arrears	none	own	false
6	arrears	none	own	true
7	current	none	own	false
8	arrears	none	own	false
9	current	none	rent	false
10	none	none	own	true
11	current	coapplicant	own	false
12	current	none	own	true
13	current	none	rent	true
14	paid	none	own	false
15	arrears	none	own	false
16	current	none	own	false
17	arrears	coapplicant	rent	false
18	arrears	none	free	false
19	arrears	none	own	false
20	paid	none	own	false

例：

$P(fr) = 0.3$	$P(\neg fr) = 0.7$
$P(CH = 'none' fr) = 0.1666$	$P(CH = 'none' \neg fr) = 0$
$P(CH = 'paid' fr) = 0.1666$	$P(CH = 'paid' \neg fr) = 0.2857$
$P(CH = 'current' fr) = 0.5$	$P(CH = 'current' \neg fr) = 0.2857$
$P(CH = 'arrears' fr) = 0.1666$	$P(CH = 'arrears' \neg fr) = 0.4286$
$P(GC = 'none' fr) = 0.8334$	$P(GC = 'none' \neg fr) = 0.8571$
$P(GC = 'guarantor' fr) = 0.1666$	$P(GC = 'guarantor' \neg fr) = 0$
$P(GC = 'coapplicant' fr) = 0$	$P(GC = 'coapplicant' \neg fr) = 0.1429$
$P(ACC = 'own' fr) = 0.6666$	$P(ACC = 'own' \neg fr) = 0.7857$
$P(ACC = 'rent' fr) = 0.3333$	$P(ACC = 'rent' \neg fr) = 0.1429$
$P(ACC = 'free' fr) = 0$	$P(ACC = 'free' \neg fr) = 0.0714$

$P(fr) = 0.3$	$P(\neg fr) = 0.7$
$P(CH = 'paid' fr) = 0.1666$	$P(CH = 'paid' \neg fr) = 0.2857$
$P(GC = 'none' fr) = 0.8334$	$P(GC = 'none' \neg fr) = 0.8571$
$P(ACC = 'rent' fr) = 0.3333$	$P(ACC = 'rent' \neg fr) = 0.1429$

$$\left(\prod_{k=1}^m P(\mathbf{q}[k] | fr) \right) \times P(fr) = 0.0139$$

$$\left(\prod_{k=1}^m P(\mathbf{q}[k] | \neg fr) \right) \times P(\neg fr) = 0.0245$$

CREDIT HISTORY	GUARANTOR/ COAPPLICANT	ACCOMODATION	FRAUDULENT
paid	none	rent	'false'

朴素贝叶斯的分类器天真地假设每个在给定目标要素的状态下，域中的描述性特征有条件地独立于所有其他描述性特征。

这个假设虽然经常是错误的，但使朴素贝叶斯模型能够最大限度地分解它使用的域表示。

令人惊讶的是，朴素贝叶斯模型通常表现相当不错。

算法进阶：

1. 平滑
2. 连续特征：概率密度函数
3. 连续功能：像素合并
4. 贝叶斯网络

1. 平滑 Smoothing

尽管条件独立性的假设扩展了朴素贝叶斯模型的覆盖范围，并允许其泛化到训练数据的内容之外，但朴素贝叶斯模型仍然不能完全覆盖所有可能的查询集。

$P(fr) = 0.3$	$P(\neg fr) = 0.7$
$P(CH = 'none' fr) = 0.1666$	$P(CH = 'none' \neg fr) = 0$
$P(CH = 'paid' fr) = 0.1666$	$P(CH = 'paid' \neg fr) = 0.2857$
$P(CH = 'current' fr) = 0.5$	$P(CH = 'current' \neg fr) = 0.2857$
$P(CH = 'arrear' fr) = 0.1666$	$P(CH = 'arrear' \neg fr) = 0.4286$
$P(GC = 'none' fr) = 0.8334$	$P(GC = 'none' \neg fr) = 0.8571$
$P(GC = 'guarantor' fr) = 0.1666$	$P(GC = 'guarantor' \neg fr) = 0$
$P(GC = 'coapplicant' fr) = 0$	$P(GC = 'coapplicant' \neg fr) = 0.1429$
$P(ACC = 'own' fr) = 0.6666$	$P(ACC = 'own' \neg fr) = 0.7857$
$P(ACC = 'rent' fr) = 0.3333$	$P(ACC = 'rent' \neg fr) = 0.1429$
$P(ACC = 'free' fr) = 0$	$P(ACC = 'free' \neg fr) = 0.0714$

【尽管已经拆分成单特征概率，还是有很多缺失值】

避免此问题的标准方法是使用平滑处理。

平滑从具有大量概率份额的事件中获取一些概率，并将其提供给集合中的其他概率。

$$P(f = v|t) = \frac{\text{count}(f = v|t) + k}{\text{count}(f|t) + (k \times |\text{Domain}(f)|)}$$

k 值越大，意味着平滑程度越高，也就是说，从较大的概率中获取质量并赋予较小的概率的概率就越大。

通常 k 采用较小的值，例如 1、2 或 3。【下例 k=3】

$P(fr) = 0.3$	$P(fr) = 0.3$
$P(CH = 'none' fr) = 0.1666$	$P(CH = 'none' fr) = 0.2222$
$P(CH = 'paid' fr) = 0.1666$	$P(CH = 'paid' fr) = 0.2222$
$P(CH = 'current' fr) = 0.5$	$P(CH = 'current' fr) = 0.3333$
$P(CH = 'arrear' fr) = 0.1666$	$P(CH = 'arrear' fr) = 0.2222$
$P(GC = 'none' fr) = 0.8334$	$P(GC = 'none' fr) = 0.5333$
$P(GC = 'guarantor' fr) = 0.1666$	$P(GC = 'guarantor' fr) = 0.2667$
$P(GC = 'coapplicant' fr) = 0$	$P(GC = 'coapplicant' fr) = 0.2$
$P(ACC = 'own' fr) = 0.6666$	$P(ACC = 'own' fr) = 0.4667$
$P(ACC = 'rent' fr) = 0.3333$	$P(ACC = 'rent' fr) = 0.3333$
$P(ACC = 'free' fr) = 0$	$P(ACC = 'Free' fr) = 0.2$

Before smoothing

After smoothing

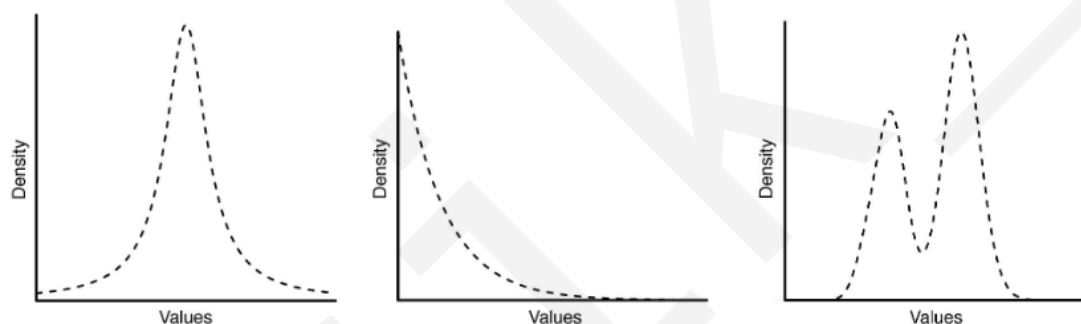
2. 连续特征：概率密度函数

Probability Density Function (PDF)

PDF 定义了密度曲线，曲线的形状由下式确定：

用于定义 PDF 的统计分布

统计分布参数的值



(a) Normal/Student-t

(b) Exponential

(c) Mixture of Gaussians

Student-T 分布 $f(x) = \frac{\Gamma(\frac{\kappa+1}{2})}{\sqrt{\kappa\pi}\Gamma(\frac{\kappa}{2})} (1 + \frac{x^2}{\kappa})^{-\frac{(\kappa+1)}{2}}$

student-t 分布围绕单个峰是对称的。

student-t 概率密度函数的定义使用 gamma 函数 Γ ，这是一个标准的统计

函数。 $\Gamma(k) = \int_0^{+\infty} t^{k-1} e^{-t} dt$

参数 κ 是分布的自由度。

对于 student-t 分布，自由度始终设置为样本数量减去 1。

student-t 分布和正态分布都是位置尺度 (location-scale) 分布族的成员。

从分布的角度来看，正态分布和 student-t 之间的主要区别在于正态分布具有轻尾，而 student-t 分布具有肥尾 (重尾)。



图 6.4: 两个单峰数据集的直方图: (a) 分布有轻尾; (b) 分布有肥/重的尾巴。

许多数据分析师常犯的一个错误是自动默认使用正态分布对单峰分布数据进行建模。

有一些统计检验（例如 KolmogorovSmirnov 检验）可用于检查特征是否呈正态分布，在特征不是正态分布的情况下，另一种单峰分布（例如 student-t 分布）可能更适合。

具有轻尾的正态分布的另一个结果是它对数据中的异常值很敏感。

student-t 对异常值的鲁棒性是考虑使用此分布（而不是正态分布）在数据集相对较小或可能嘈杂的情况下对单峰数据进行建模的另一个原因。

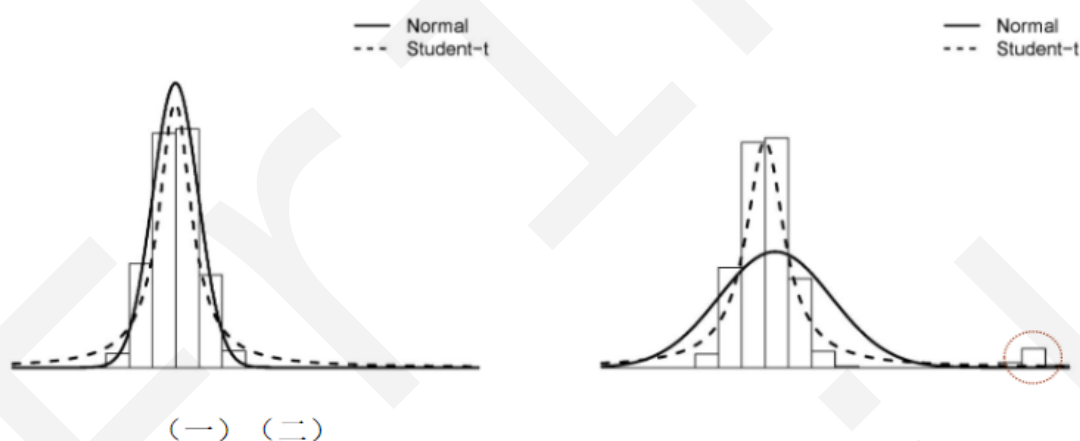


图 6.5: student-t 分布对异常值的鲁棒性说明：(a) 单峰数据集的密度直方图，叠加了拟合到数据的正态分布和 student-t 分布的密度曲线；(b) 添加了异常值的同一数据集的密度直方图，与拟合到数据的正态分布和 student-t 分布的密度曲线重叠。

student-t 分布受异常值引入的影响较小。

$$\text{指数分布 } E(x, \lambda) = \begin{cases} \lambda e^{-\lambda x} & \text{for } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

指数分布采用一个参数 λ ，称为速率。

改变 λ 值会改变密度下降的速率。 λ 变大，分布的峰值变大，下降变陡。

为了将指数分布拟合到连续特征，我们将 λ 设置为等于 1 除以特征的平

$$\begin{aligned} \bar{m} &= \int_0^{\infty} xp(x)dx \\ \text{均值。} &= \int_0^{\infty} \lambda xe^{-\lambda x} dx \\ &= \frac{1}{\lambda} \end{aligned}$$

通常用于对等待时间进行建模： λ 等于 1 除以事件所需的平均时间。

$$\text{高斯分布混合 } N(x, \mu_1, \sigma_1, \omega_1, \dots, \mu_n, \sigma_n, \omega_n) = \sum_{i=1}^n \frac{\omega_i}{\sigma_i \sqrt{2\pi}} e^{-\frac{(x-\mu_i)^2}{2\sigma_i^2}}$$

高斯分布的混合用于表示由多个子群体组成的数据。

密度曲线中的多个峰来自不同的亚群（具有多个峰的分佈称为多峰）。

使用高斯分布的混合假设数据中的所有子种群都按照正态分布分布，但**这些子群正态分布中的每一个都具有不同的均值，并且可能具有不同的标准差。**

总和中组分的重量决定了该组分对所得混合物总密度的贡献。

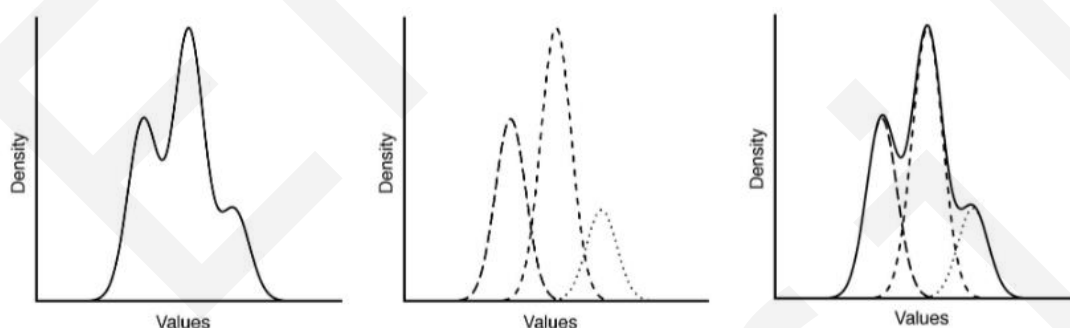
高斯分布的混合由每个分量的三个参数定义：

均值 μ

标准差 σ

权重 ω

混合物的重量参数集之和必须为 1



PDF 是对密度直方图的抽象，因此 PDF 表示曲线下面积的概率。

要使用 PDF 计算概率，我们需要根据 PDF 曲线间隔下的面积来考虑。

我们可以通过在概率表中查找来计算 PDF 下的面积，或者使用积分来计算区间范围内曲线下的面积。

流程：

选取模型

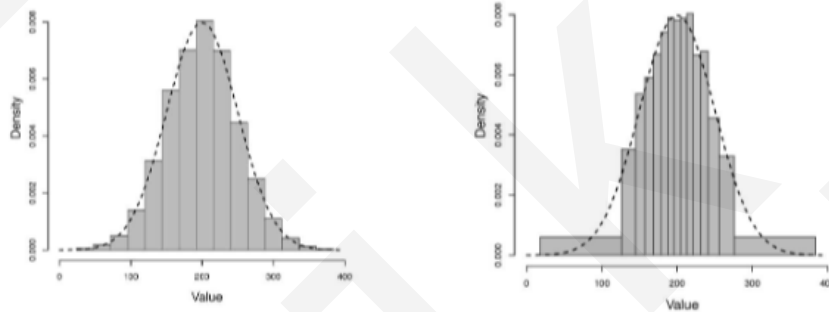
拟合数据

计算参数

应用计算

3. 连续功能：像素合并

- In Section 3.6.2 we explained two of the best known binning techniques **equal-width** and **equal-frequency**.



- These techniques can be used to bin continuous features into categorical features
- In general we recommend **equal-frequency binning**.

ID	LOAN AMOUNT	BINNED LOAN AMOUNT	FRAUD	ID	LOAN AMOUNT	BINNED LOAN AMOUNT	FRAUD
15	500	bin1	false	9	20,000	bin3	false
19	500	bin1	false	7	25,000	bin3	false
1	900	bin1	true	5	32,000	bin3	false
10	9,500	bin1	true	20	35,000	bin3	false
12	9,850	bin1	true	3	48,000	bin3	false
4	10,000	bin2	true	18	50,000	bin4	false
17	15,450	bin2	false	14	65,000	bin4	false
16	16,000	bin2	false	13	95,500	bin4	true
11	16,750	bin2	false	2	150,000	bin4	false
8	18,500	bin2	false	6	250,000	bin4	true

所有 bin3 值的目标特征值均为 false。因此，以 $FRAUD = true$ 为条件的 $LOAN\ AMOUNT = bin3$ 的后验概率为 0.0，以 $LOAN\ AMOUNT = bin3$ 为条件的 $FRAUD = false$ 的后验概率为 1.0。

平滑应与分箱结合使用，以帮助处理极端概率。

4. 贝叶斯网络

贝叶斯网络使用基于图的表示来编码域中特征子集之间的结构关系，例如直接影响和条件独立性。

因此，贝叶斯网络表示通常比完全联合分布更紧凑，但不会强制在所有描述性特征之间断言全局条件独立性。

贝叶斯网络是一个有向无环图，由三个基本元素组成：

节点

域中的每个要素都由图中的单个节点表示。

边缘

节点通过定向链接连接；

图中链接的连通性对节点之间的影响和条件独立性关系进行编码。

条件概率表（CPT）

每个节点都有一个与之关联的 CPT。

CPT 列出了节点所表示的特征的概率分布，该特征的条件是节点通过边连接到的其他节点所表示的特征。

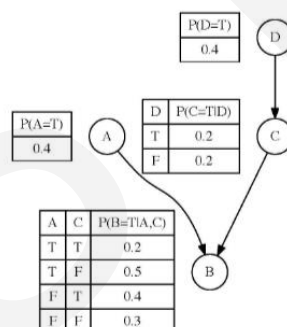
对于任何具有 N 个节点的网络，事件 $x_1, \dots, x_n$ 的概率可以使用以下公式计算：

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{Parents}(x_i))$$

其中 $\text{Parents}(x)$ 描述图中直接链接到节点 x 的一组节点。

使用这个方程，我们可以计算贝叶斯网络所表示的域中的任何联合事件。

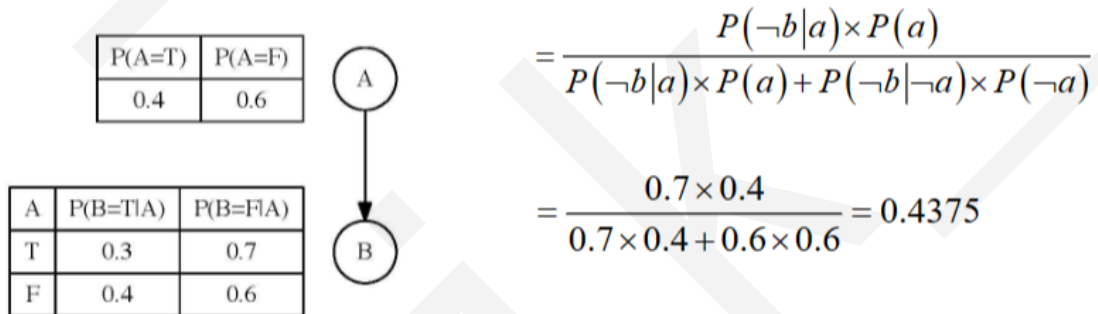
$$\begin{aligned} P(a, \neg b, \neg c, d) &= P(\neg b | a, \neg c) \times P(\neg c | d) \times P(a) \times P(d) \\ &= (1 - 0.5) \times (1 - 0.2) \times 0.4 \times 0.4 = 0.064 \end{aligned}$$



我们可以使用贝叶斯定理来反转网络中节点之间的依赖关系。

$$P(a|\neg b) = \frac{P(\neg b|a) \times P(a)}{P(\neg b)} = \frac{P(\neg b|a) \times P(a)}{\sum_i P(\neg b|A_i) \times P(A_i)}$$

P288



对于有条件的独立性，我们不仅需要考虑节点的父节点，还需要考虑其子节点及其父节点的状态。

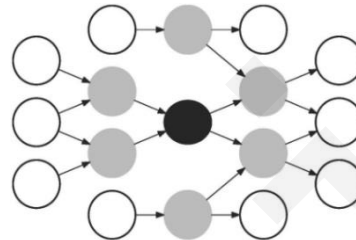
图中使节点独立于图的其余部分的节点集称为节点的**马尔可夫毯**。

灰色节点定义了黑色节点的马尔可夫毯。给定灰色节点的状态，黑色节点有条件地独立于白色节点。

【这是一个近似的公式】

【得数不完全与实际概率一致】

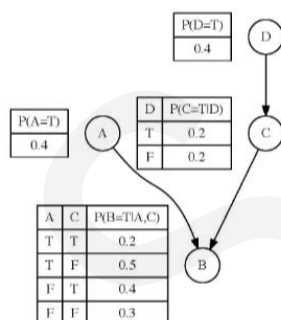
【只有系数上的区别】



$$P(x_i | x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) = P(x_i | \text{Parents}(x_i)) \prod_{x_j \in \text{Children}(x_i)} P(x_j | \text{Parents}(x_j))$$

例：

$$P(c|\neg a, b, d) = P(c|d) \times P(b|c, \neg a) = 0.2 \times 0.4 = 0.08$$



朴素贝叶斯【去掉正则化，加上条件独立后的公式】

$$P(t=l|\mathbf{q}[1], \dots, \mathbf{q}[m]) = \frac{P(\mathbf{q}[1], \dots, \mathbf{q}[m]|t=l)P(t=l)}{P(\mathbf{q}[1], \dots, \mathbf{q}[m])} \quad (6.8)$$

without
normalization

$$\Rightarrow P(\mathbf{q}[1], \dots, \mathbf{q}[m]|t=l)P(t=l)$$

$$= P(\mathbf{q}[1]|t=l) \times P(\mathbf{q}[2]|\mathbf{q}[1], t=l) \times \dots \times P(\mathbf{q}[m]|\mathbf{q}[m-1], \dots, \mathbf{q}[2], \mathbf{q}[1], t=l) \times P(t=l)$$

conditional
independence

$$\Rightarrow P(\mathbf{q}[1]|t=l) \times P(\mathbf{q}[2]|t=l) \times \dots \times P(\mathbf{q}[m]|t=l) \times P(t=l)$$

$$= P(t=l) \prod_{i=1}^m P(\mathbf{q}[i]|t=l)$$

$$P(t=l|\mathbf{q}[1], \dots, \mathbf{q}[m]) \Rightarrow P(t=l) \prod_{i=1}^m P(\mathbf{q}[i]|t=l)$$

朴素贝叶斯分类器是具有特定拓扑结构的贝叶斯网络。

【从一个根事件出发指向其他所有事件】

未知父节点

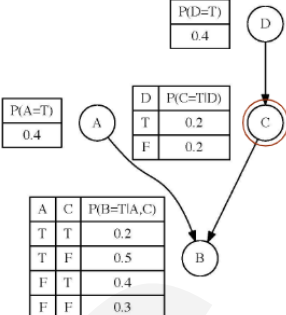
如果一个或多个父节点的值未知，则计算节点的条件概率会变得更加复杂。

在这种情况下，节点将依赖于其未知父节点的祖先。

这是因为如果父节点未知，那么要计算该节点的分布，我们必须对这个父节点求和。

但是，要进行这种求和，我们必须知道未知父项的分布，这反过来又要求我们求和父项的父项，必要时依此类推。

▪ Compute $P(b|a, d)$ by summing out C:



$$P(b|a, d) = \frac{P(b, a, d)}{P(a, d)} = \frac{\sum_i P(b, a, C_i, d)}{\sum_i P(a, C_i, d)}$$

$$= \frac{P(b|a, c) \times P(a) \times P(c|d) \times P(d) + P(b|a, -c) \times P(a) \times P(-c|d) \times P(d)}{P(a) \times P(c|d) \times P(d) + P(a) \times P(-c|d) \times P(d)}$$

$$= \frac{P(b|a, c) \times P(a) \times P(c|d) + P(b|a, -c) \times P(a) \times P(-c|d)}{P(a) \times P(c|d) + P(a) \times P(-c|d)}$$

$$= \frac{0.2 \times 0.4 \times 0.2 + 0.5 \times 0.4 \times 0.8}{0.4 \times 0.2 + 0.4 \times 0.8} = 0.44$$

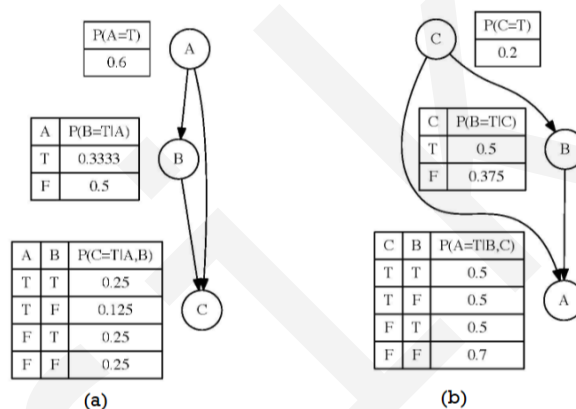
当无法完全了解网络中所有节点的状态时，我们会限制已知节点的值，并对未知节点求和。

此外，在这些计算过程中，我们只需要在马尔可夫毯上调节一个节点，这大大减少了网络所需的概率。

贝叶斯网络可以手动构建，也可以从数据中学习。

学习贝叶斯网络的拓扑结构和网络中 CPT 中的参数是一项艰巨的计算任务。

学习贝叶斯网络结构如此困难的原因之一是，可以将几个不同的贝叶斯网络定义为同一完整联合概率分布的表示。



这两个网络编码相同的联合概率分布

不同网络结构算出来的条件概率可能一样可能不一样，但是一定是成比例的。事件之间的概率大小关系不会发生变化。

与其他预测模型类型相比，贝叶斯网络的一个真正优势是，当查询实例中缺少一个或多个描述性特征值时，它们提供了一种优雅的方案来预测目标特征。并不强制在所有描述性特征之间断言全局条件独立性。

由于使用贝叶斯网络进行精确推理时可能会产生计算复杂性，因此一种流行的方法是使用马尔可夫链蒙特卡罗算法来近似所需的概率分布。

吉布斯采样是最著名的 MCMC 算法之一。

限制证据变量的值并随机分配非证据变量的值。

通过使用以网络其余部分的状态为条件的节点分布来更改其中一个非证据变量的值来生成样本。

第七章：基于误差的学习

在基于错误的机器学习中，我们为参数化模型搜索一组参数，该模型对一组训练实例所做的预测的总误差最小化。

参数化预测模型使用一组随机参数进行初始化，并使用误差函数来判断该初始模型在对训练数据集中的实例进行预测时的性能。

根据误差函数的值，对参数进行迭代调整，以创建越来越准确的模型。

对于测量简单线性回归模型的拟合，最常用的是误差函数的平方和，即 L

$$L_2(\mathbf{M}_w, D) = \frac{1}{2} \sum_{i=1}^n (t_i - (\mathbf{w}[0] + \mathbf{w}[1] \times \mathbf{d}_i[1]))^2$$

x - y 平面称为权重空间，曲面称为误差曲面。

最适合训练数据的模型是与误差面上的最低点对应的模型。

在误差面上正式定义该点：

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}[0]} \frac{1}{2} \sum_{i=1}^n (t_i - (\mathbf{w}[0] + \mathbf{w}[1] \times \mathbf{d}_i[1]))^2 &= 0 \\ \frac{\partial}{\partial \mathbf{w}[1]} \frac{1}{2} \sum_{i=1}^n (t_i - (\mathbf{w}[0] + \mathbf{w}[1] \times \mathbf{d}_i[1]))^2 &= 0 \end{aligned}$$

标准方法：具有梯度下降的多元线性回归

定义权重向量 $\mathbf{w}$ ，将损失函数写为：

$$\begin{aligned} L_2(\mathbf{M}_w, D) &= \frac{1}{2} \sum_{i=1}^n (t_i - \mathbf{M}_w(\mathbf{d}_i))^2 \\ &= \frac{1}{2} \sum_{i=1}^n (t_i - \mathbf{w} \cdot \mathbf{d}_i)^2 \end{aligned}$$

1. $\mathbf{w} \leftarrow$ random starting point in the weight space
2. **repeat**
3. **for** each $\mathbf{w}[j]$ in $\mathbf{w}$ **do**
4. $\mathbf{w}[j] \leftarrow \mathbf{w}[j] + \alpha \times \text{errorDelta}(D, \mathbf{w}[j])$
5. **end for**
6. **until** convergence occurs

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbf{M}_w, D) &= \frac{\partial}{\partial \mathbf{w}[j]} \left[\frac{1}{2} (t - \mathbf{M}_w(\mathbf{d}))^2 \right] \\
&= (t - \mathbf{M}_w(\mathbf{d})) \times \frac{\partial}{\partial \mathbf{w}[j]} (t - \mathbf{M}_w(\mathbf{d})) \\
&= (t - \mathbf{M}_w(\mathbf{d})) \times \frac{\partial}{\partial \mathbf{w}[j]} (t - \mathbf{w} \cdot \mathbf{d}) \\
&= (t - \mathbf{M}_w(\mathbf{d})) \times (-\mathbf{d}[j])
\end{aligned}$$

$$\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbf{M}_w, D) = \sum_{i=1}^n (t_i - \mathbf{M}_w(\mathbf{d}_i)) \times (-\mathbf{d}_i[j])$$

$$\begin{aligned}
errorDelta(D, \mathbf{w}[j]) &= -\frac{\partial}{\partial \mathbf{w}[j]} L_2(\mathbf{M}_w, D) \\
&= \sum_{i=1}^n (t_i - \mathbf{M}_w(\mathbf{d}_i)) \times \mathbf{d}_i[j]
\end{aligned}$$

$$\mathbf{w}[j] \leftarrow \mathbf{w}[j] + \alpha \underbrace{\sum_{i=1}^n (t_i - \mathbf{M}_w(\mathbf{d}_i)) \times \mathbf{d}_i[j]}_{errorDelta(D, \mathbf{w}[j])}$$

学习率 α ，决定了在过程中的每个步骤中对每个权重所做的调整的大小。

根据经验证据，从 $[-0.2, 0.2]$ 范围内均匀地选择随机初始权重效果很好。

算法进阶：

1. 解释多变量线性回归模型
2. 使用权重衰减设置学习率
3. 处理分类描述性特征
4. 处理分类目标特征：逻辑回归
5. 非线性关系建模
6. 多项式 Logistic 回归
7. 支持向量机

1. 解释多变量线性回归模型

权重的直接比较并不能告诉我们它们的相对重要性。

确定模型中每个描述性特征重要性的更好方法是执行统计学显著性检验。

- 计算 test-statistic。
- 计算检验统计量值大于或大于计算值的概率。此概率称为 p 值。
- 将 p 值与预定义的显著性阈值进行比较，如果 p 值小于或等于阈值（即 p 值较小），则原假设将被拒绝。这些阈值通常是 5% 或 1% 的标准统计阈值。

使用 t 检验来分析描述性特征 $d[j]$ 在线性回归模型中的重要性。

此检验的原假设是特征对模型没有显著影响。

- The standard error for the overall model is calculated as

$$se = \sqrt{\frac{\sum_{i=1}^n (t_i - M_w(d_i))^2}{n-2}} \quad (7.18)$$

- A standard error calculation is then done for a descriptive feature as follows:

$$se(d[j]) = \frac{se}{\sqrt{\sum_{i=1}^n (d_i[j] - \overline{d[j]})^2}} \quad (7.19)$$

- The t-statistic for this test is calculated as follows:

$$t = \frac{w[j]}{se(d[j])} \quad (7.20)$$

如果 p 值小于所需的显著性水平（通常为 0.05），则我们否定原假设，并说描述性特征对模型有显著影响

2. 使用权重衰减设置学习率

学习率衰减允许学习率从较大的值开始，然后根据预定义的时间表随时间推移而衰减。

$$\alpha_{\tau} = \alpha_0 \frac{c}{c + \tau}$$

使用学习率衰减甚至可以解决学习率过大导致误差平方和增加而不是减少的问题。

尽管学习率衰减几乎总是比固定学习率带来更好的性能，但它仍然需要为 α 和 c 选择与问题相关的值。

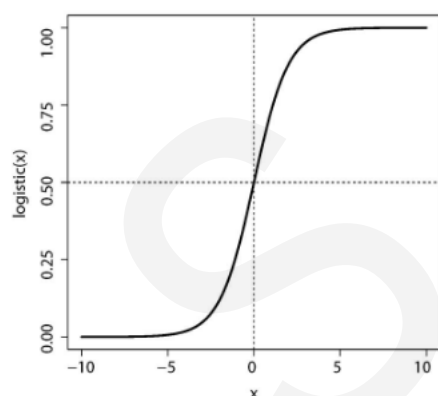
3. 处理分类描述性特征

处理分类特征的最常见方法是使用转换，将单个分类描述性特征转换为多个连续的描述性特征值，这些值可以对分类特征的级别进行编码。

ENERGY RATING	R	ENERGY RATING A	ENERGY RATING B	ENERGY RATING C
C	0	0	1	1
A	1	0	0	0
A	1	0	0	0
B	0	1	0	0
C	0	0	1	1
B	0	1	0	0
B	0	1	0	0
A	1	0	0	0
C	0	0	1	1
B	0	1	0	0

4. 处理分类目标特征：逻辑回归

我们通过使用更复杂的阈值函数来解决这些问题，该函数是连续的，因此是可微分的，并且允许所需的可微可导：逻辑斯蒂函数



$$\text{Logistic}(x) = \frac{1}{1+e^{-x}}$$

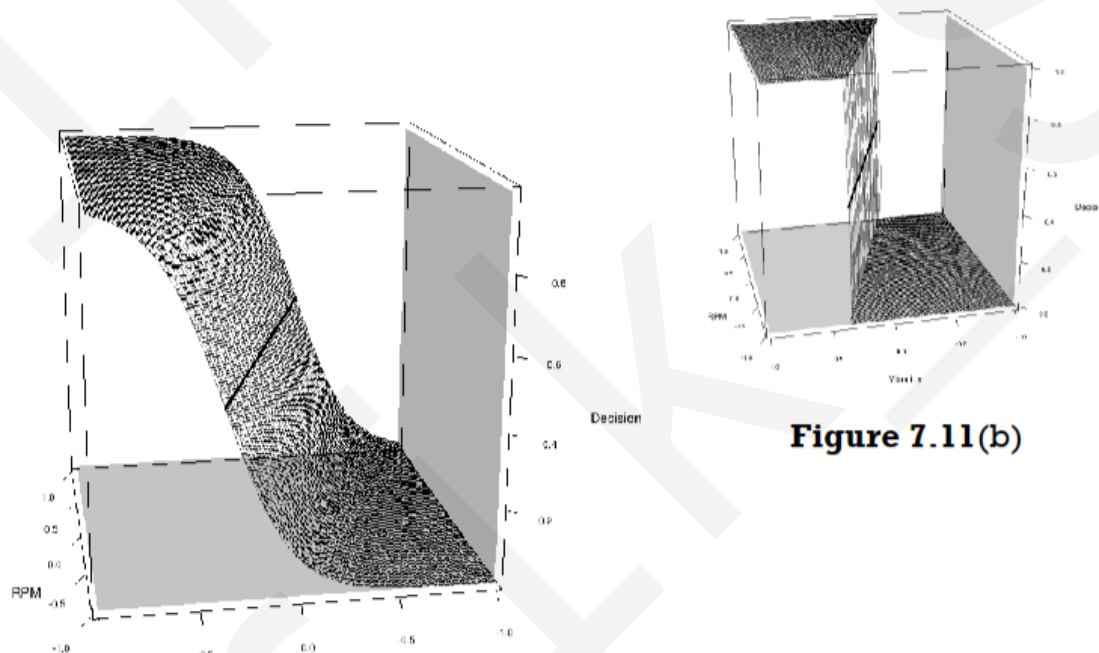


Figure 7.11(b)

使用 logistic 函数的一个好处是，逻辑回归模型输出可以解释为目标水平出现的概率

为了重新调整梯度下降算法的用途来训练逻辑回归模型，唯一需要进行的更改是权重更新规则。

对于逻辑回归模型，我们建议始终对描述性特征值进行归一化。

在训练过程开始之前，描述性特征都归一化为范围 $[-1, 1]$ 。

5. 非线性关系建模

为了处理非线性关系，我们使用一组基函数集合来转换数据而不是模型

每个基函数都以不同的方式转换输入向量 d 。

这样做的好处是，除了引入基函数机制外，我们不需要对到目前为止提出的方法进行任何其他更改。

基函数还可用于训练涉及非线性关系的分类预测问题的逻辑回归模型。

随着基函数的数量超过描述性特征的数量，模型的复杂性增加，因此梯度下降过程必须搜索更复杂的权重空间。与逻辑回归模型相比，它们的可解释性不强，尤其是在使用核函数时，很难理解为什么会做出特定的预测

我们必须手动决定使用哪一组基础函数，权重的数量通常远远大于描述性特征的数量，因此找到最佳权重集需要在更大的权重空间中进行搜索。

6. 多项式 Logistic 回归

构建多项式逻辑回归模型的一个好方法是使用一组一对多模型。

如果我们有 r 个目标水平，我们创建 r 个一对多的逻辑回归模型。区分目标要素的一个 level 和所有其他 level。

使用的 r 组一对多逻辑回归模型是并行训练的，在训练过程中计算每个模型的平方误差之和时，将使用修订后的模型输出。

【输出的概率预测需要归一化】

7. 支持向量机

从决策边界到最近的训练实例的距离称为边距。

决策边界两侧的虚线表示边距的范围，我们将其称为边距范围。

训练支持向量机涉及搜索决策边界或分离超平面，从而获得最大余量，因为这将最好地分离目标特征的级别。

训练数据集中【沿边距范围落下】的实例（因此定义边距）称为支持向量并定义决策边界。

- The required **constraints** required by the training process are

$$w_0 + \mathbf{w} \cdot \mathbf{d} \leq -1 \quad \text{for } t_i = -1 \quad (7.42)$$

and:

$$w_0 + \mathbf{w} \cdot \mathbf{d} \geq +1 \quad \text{for } t_i = +1 \quad (7.43)$$

- We can combine these two constraints into a single constraint (remember t_i is always equal to either -1 or +1):

$$t_i \times (w_0 + \mathbf{w} \cdot \mathbf{d}) \geq 1 \quad (7.44)$$

对于沿边距范围的实例， $\text{abs}(\mathbf{w} \cdot \mathbf{d} + w_0) = 1$

从沿边距范围到决策边界的任何实例的距离为 $1/|\mathbf{w}|$ ，并且由于边距与决策边界的任一侧对称，因此边距的大小为 $2/|\mathbf{w}|$ 。

可以快速训练，不太容易受到过拟合的影响，并且适用于高维数据。

与逻辑回归模型相比，它们的可解释性不强，尤其是在使用核函数时，很难理解为什么会做出特定的预测

对过拟合具有鲁棒性，并且在处理非常高维的问题时表现良好

第八章：深度学习

深度学习描述了对现代人工神经网络的研究

人工神经网络模型由大量简单的处理单元（称为神经元）组成，这些处理单元通常排列成层并高度相互连接。

人脑的复杂行为源于大量高度连接的神经元之间的相互作用，而不是神经元内部的复杂处理。

深度学习模型被称为人工神经网络。

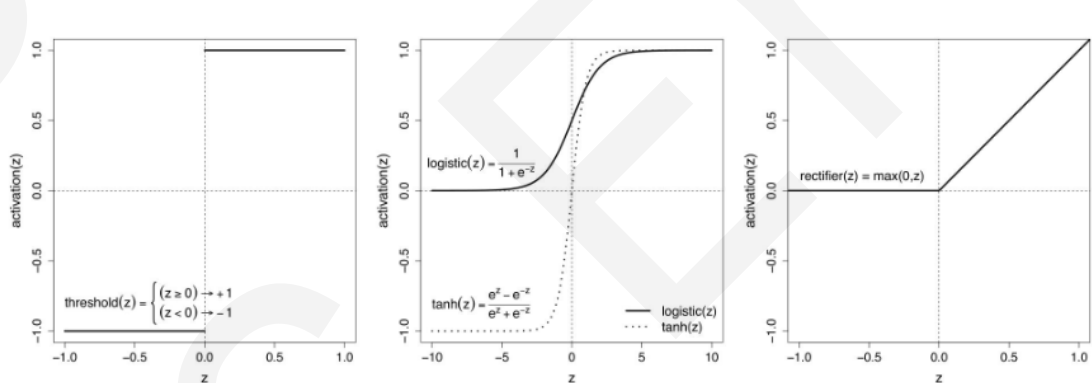
在 MP 模型的第一阶段，每个输入乘以一个权重，然后将这些乘法的结果相加。这种计算称为加权总和。

$$\begin{aligned}
 z &= \underbrace{\mathbf{w}[0] \times \mathbf{d}[0] + \mathbf{w}[1] \times \mathbf{d}[1] + \cdots + \mathbf{w}[m] \times \mathbf{d}[m]}_{\text{weighted sum}} \\
 &= \sum_{j=0}^m \mathbf{w}[j] \times \mathbf{d}[j] \\
 &= \underbrace{\mathbf{w} \cdot \mathbf{d}}_{\text{dot product}} = \underbrace{\mathbf{w}^T \mathbf{d}}_{\text{matrix product}} = [w_0 \quad w_1 \quad \cdots \quad w_m] \begin{bmatrix} d_0 \\ d_1 \\ \vdots \\ d_m \end{bmatrix}
 \end{aligned}$$

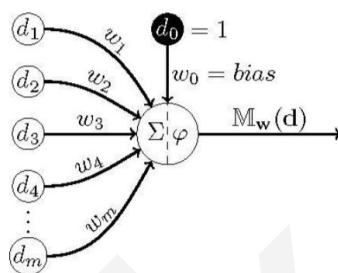
在 MP 模型的第二阶段，通过将 z 的值与阈值 θ 进行比较，将加权和计算的结果 z 转换为高激活或低激活：

然而，目前最流行的激活函数选择是整流线性激活函数或整流器

使用整流器功能的神经元称为**整流线性单元或 ReLU**



在神经元定义的输入到输出映射中引入非线性，使人工网络能够学习复杂的非线性映射。



人工神经网络可以具有任何结构，但基于层的神经元组织很常见。

传感神经元和处理神经元 sensing neurons and processing neurons

传感神经元允许网络感知外部输入，呈现给网络的输入不会被传感神经元转换，处理神经元使用加权和后跟激活函数的两步过程转换其输入

在前馈网络中，网络连接中没有允许神经元的输出作为输入流回神经元的循环或循环。该网络也是一个完全连接的网络

输入通过感应层中的神经元呈现给网络；网络对输入的响应是最终输出层中神经元的激活；内部层称为隐藏层

神经网络的深度等于隐藏层数加上输出层数

证明具有三层处理神经元的网络可以近似任何函数到任意精度

神经网络作为矩阵运算

将神经网络表示为矩阵运算序列提供了网络深度的透明视图：

网络深度等于具有与其关联的权重矩阵的层数。

加快网络中每一层的加权和的计算速度

使网络能够并行处理示例

允许使用称为图形处理单元（GPU）的专用硬件

为什么需要非线性激活函数？

无论网络中引入了多少隐藏层，仅使用线性神经元的多层前馈神经网络都等同于具有线性神经元的单层网络

为了创建可以表示复杂非线性函数的网络，仅仅添加层是不够的，我们还必须包括这些层之间的非线性。

为什么网络深度很重要？

通常情况下，使用更深层次的网络可以大大减少使网络能够表示目标功能所需的神经元数量，增加网络深度是有代价的，例如梯度消失问题，这会减慢网络的学习速度

标准方法：反向传播和梯度下降

隐藏神经元的计算需要首先计算所有下游神经元

$$\frac{\partial \varepsilon}{\partial w_{i,k}} = \frac{\partial \varepsilon}{\partial a_i} \times \frac{\partial a_i}{\partial z_i} \times \frac{\partial z_i}{\partial w_{i,k}}$$

- Working from a network weight $w_{i,k}$ forward toward the network error, we have:
 - The rate of change of the weighted sum function with respect to changes in one of the weights;
 - The rate of change of the activation function with respect to changes in the weighted sum;
 - The rate of change of the network error with respect to changes in the activation function.

随机梯度下降和批量梯度下降之间的区别可用于阐明神经网络训练中经常使用的两个相关术语的含义：epoch 和 iteration

算法进阶：

1. 梯度消失和 ReLU
2. 处理分类目标特征：Softmax 输出层和交叉熵损失函数
3. 卷积神经网络

1. 梯度消失和 ReLU

梯度消失是训练深度网络的一个严峻挑战，因为梯度是学习信号，它告诉每个神经元应该如何更新其权重以提高网络的性能，网络越深，早期层的学习速度就越慢

梯度消失问题是反向传播算法基于链式法则这一事实的直接结果
解决梯度消失问题的一种方法是使用不同的激活函数

例如，整流线性函数 ReLU

将网络从逻辑激活函数切换到整流线性激活函数可以加快训练速度

它还倾向于使网络学习的表示变得稀疏，这意味着对于任何输入向量，只有网络中神经元的正子集会激活（即 $a > 0$ ），输入向量流经网络中活动通路的子集

ReLU 在所有输入中都处于非活动状态，因此从不更新，这种动态被称为垂死的 ReLU 问题，如果网络中有太多的神经元死亡，则网络在训练期间不会收敛。避免死 ReLU 的一种方法是将网络的所有偏置权重初始化为一个小的正值，例如 0.1。

泄漏的 ReLU：Leaky ReLU

参数化 ReLU：Parametric ReLU

2. 处理分类目标特征：Softmax 输出层和交叉熵损失函数

要创建可以预测多级分类特征的神经网络，需要进行三项调整：

使用独热编码表示目标要素

将网络的输出层更改为 softmax

将训练的误差（或损失）函数更改为交叉熵函数

独热编码：它被称为独热表示，因为向量中最多有一个元素的值为 1，所有其他元素的值为 0，特征的值由哪个元素表示为 1

在 softmax 输出层中，目标特征的每个级别都有一个神经元。

$$\varphi_{sm}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^m e^{z_m}}$$

softmax 激活函数在层中返回一组归一化的正值这一事实使我们能够将层中每个神经元的激活解释为概率，预测分类目标特征的某个级别的概率

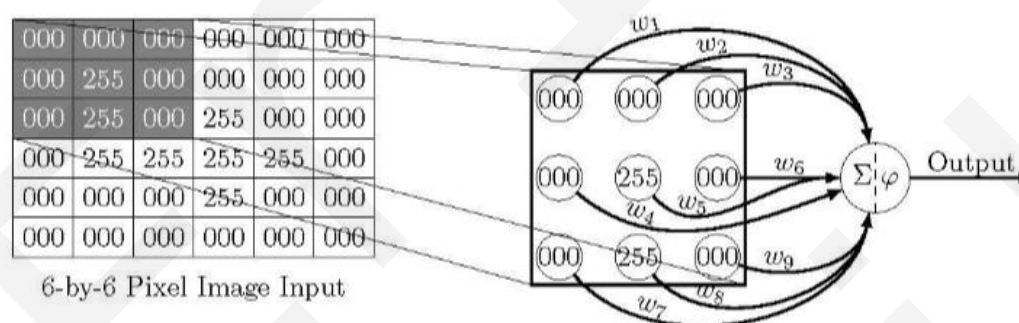
交叉熵： $L_{CE}(\mathbf{t}, \hat{\mathbf{P}}) = -\sum_j t_j \ln(\hat{P}_j)$

交叉熵损失衡量真实分布 t 与预测分布之间的相异性

$$\begin{aligned}
 L_{CE}(\mathbf{t}, \hat{\mathbf{P}}) &= -\sum_j t_j \ln(\hat{P}_j) \\
 &= -\left((t_0 \ln(\hat{P}_0)) + (t_1 \ln(\hat{P}_1)) + (t_2 \ln(\hat{P}_2)) \right) \\
 &= -\left((0 \ln(\hat{P}_0)) + (1 \ln(\hat{P}_1)) + 0(t_2 \ln(\hat{P}_2)) \right) \\
 &= -\ln(\hat{P}_1)
 \end{aligned} \tag{8.68}$$

3. 卷积神经网络 CNN

卷积神经网络结合了局部感受野、权重共享和子采样



当一些神经元共享一个过滤器时，过滤器中的每个权重在前向传递期间会多次使用

池化

在卷积神经网络中，进行子采样

许多现代卷积网络使用 $\max$ 函数，该函数仅返回局部感受野覆盖的特征图区域中的最大值

第九章：评估

标准方法：测量保持测试集的误分类率

保留测试集是通过在数据准备阶段创建的 ABT 中随机抽样部分数据来创建的，此随机样本从未在训练中使用过

使用保留测试集可以避免窥视问题，当使用用于训练模型的相同数据评估模型的性能时，会出现窥视问题

▪ For binary prediction problems there are 4 possible outcomes:

1. True Positive (TP)

- an instance in the test set that had a **positive** target feature value and that was predicted to have a **positive** target feature value

2. True Negative (TN)

3. False Positive (FP)

- an instance in the test set that had a **negative** target feature value but that was predicted to have a **positive** target feature value

4. False Negative (FN)

混淆矩阵

		Prediction	
		positive	negative
Target	positive	TP	FN
	negative	FP	TN

$$TPR = \frac{TP}{TP + FN}$$

$$TNR = \frac{TN}{TN + FP}$$

$$FPR = \frac{FP}{TN + FP}$$

$$FNR = \frac{FN}{TP + FN}$$

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN} \quad TPR = \frac{TP}{TP + FN}$$

召回率等同于真阳性率（TPR）。召回率告诉我们，模型已找到所有具有正目标水平的实例，这告诉我们要多大的可信度。

精度告诉我们，预测为具有正目标水平的实例实际上具有正目标水平的置信度。

$$F_1\text{-measure} = 2 \times \frac{precision \times recall}{precision + recall}$$

然而，在许多情况下，认为一个目标水平更重要是没有意义的。在这些情况下，平均班级精度性能测量 average class accuracy 可能是有效的。

$$average\ class\ accuracy = \frac{1}{|level(t)|} \sum_{l \in level(t)} recall_l$$

Table 9.5: A confusion matrix for a k-NN model trained on a churn prediction problem.

		Prediction	
		'non-churn'	'churn'
Target	'non-churn'	90	0
	'churn'	9	1

classification accuracy
= 91%

average class accuracy
= $\frac{1}{2}(1+0.1)$
= 55%

Table 9.6: A confusion matrix for a naive Bayes model trained on a churn prediction problem.

		Prediction	
		'non-churn'	'churn'
Target	'non-churn'	70	20
	'churn'	2	8

classification accuracy
= 78%

average class accuracy
= $\frac{1}{2}(0.778+0.8)$
= 78.889%

计算平均类精度时，调和平均值优于算术平均值。

$$average\ class\ accuracy_{HM} = \frac{1}{\frac{1}{|level(t)|} \sum_{l \in level(t)} \frac{1}{recall_l}}$$

Table 9.5: A confusion matrix for a k-NN model trained on a churn prediction problem.

		Prediction	
		'non-churn'	'churn'
Target	'non-churn'	90	0
	'churn'	9	1

average
class accuracy_{HM}
= $\frac{1}{\frac{1}{2}(\frac{1}{1.0} + \frac{1}{0.1})}$ = 18.2%

Table 9.6: A confusion matrix for a naive Bayes model trained on a churn prediction problem.

		Prediction	
		'non-churn'	'churn'
Target	'non-churn'	70	20
	'churn'	2	8

average
class accuracy_{HM}
= $\frac{1}{\frac{1}{2}(\frac{1}{0.778} + \frac{1}{0.800})}$ = 78.873%

调和平均值比算术平均值更强调较小值的贡献

一种方法是计算每个预测产生的损益，并使用它来确定模型的整体性能。

为此，我们首先需要创建一个记录这些的利润矩阵。

利润难以准确度量

绩效度量：预测分数

有许多基于比较预测分数分布的想法的性能度量，试图满足真实数据的特殊性。

受试者工作特征曲线（ROC 曲线）

Kolmogorov-Smirnov 统计量（K-S 统计）

测量增益 Gain 和提升 Lift

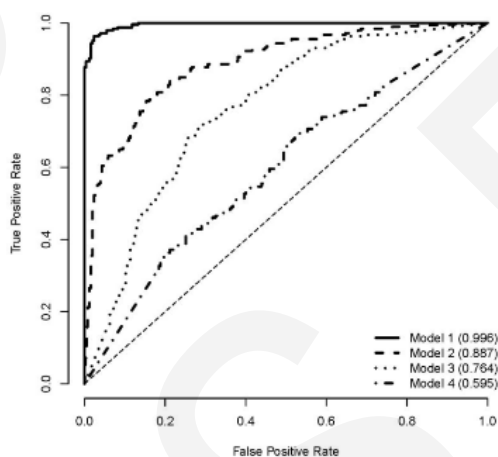
ROC:

TPR 和 TNR 与用于将预测分数转换为目标水平的阈值有着内在的联系。

为了绘制 ROC 曲线，我们创建了一个图表，在垂直访问上具有真阳性率

（TPR），在水平轴上具有假阳性率（FPR）（或 $1 - \text{TNR}$ ）。

我们期望经过训练的模型的 ROC 曲线高于随机参考线，随着预测模型强度的增加，ROC 曲线会远离随机线，向 ROC 空间的左上角移动，TPR 为 1，FPR 为 0。曲线越接近左上角，模型的预测性就越强。



ROC index 衡量 ROC 曲线下方的面积。高于 0.7 的值表示强模型，而低于 0.6 的值表示弱模型

基尼系数是 ROC 指数的线性重缩放

$$Gini\ coefficient = (2 \times ROCindex) - 1$$

K-S 统计量，范围从 0 到 1，值越高表示模型性能越好，这反映了模型预测的负实例和正实例的分数分布之间存在明显区别。

在某些情况下，只对正目标水平特别感兴趣，关注模型对这些实例的预测程度，而不是模型区分两个目标水平的能力，增益和提升通常很有用。

第十四章：Art of MLPDA

机器学习≈归纳学习

1. 通过归纳法学习的模型不能保证是正确的。
2. 除非学习过程在某种程度上存在偏好，否则学习就无法发生。

$$H(t, D) = - \sum_{l \in \text{levels}(t)} \left(P(t = l) \times \log_s (P(t = l)) \right)$$

$$\text{dist}(\mathbf{q}, \mathbf{d}) = \sqrt{\sum_{i=1}^m (\mathbf{q}[i] - \mathbf{d}[i])^2}$$

$$P(t = l | \mathbf{q}) = \frac{P(\mathbf{q} | t = l) P(t = l)}{P(\mathbf{q})}$$

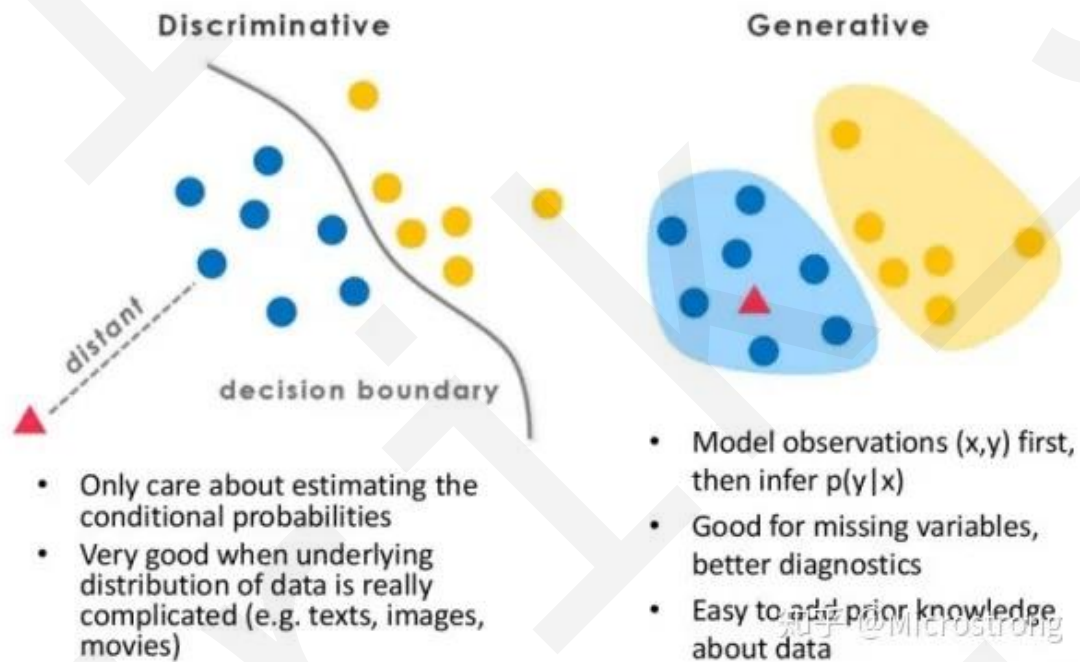
$$L_2(\mathbf{M}_w, D) = \frac{1}{2} \sum_{i=1}^n (t_i - \mathbf{M}_w(\mathbf{d}_i))^2$$

$$\frac{\partial \varepsilon}{\partial w_{i,k}} = \frac{\partial \varepsilon}{\partial a_i} \times \frac{\partial a_i}{\partial z_i} \times \frac{\partial z_i}{\partial w_{i,k}}$$

Model	Parametric/ Non-Parametric	Generative/ Discriminative
k nearest neighbor	Non-Parametric	Generative
Decision Trees	Non-Parametric	Discriminative
Naive Bayes	Parametric	Generative
Bayesian Network	Parametric	Generative
Linear Regression	Parametric	Discriminative
Logistic Regression	Parametric	Discriminative
SVM	Non-Parametric	Discriminative
Neural networks	Parametric ¹	Discriminative

神经网络被归类为参数模型;然而,可以说深度网络是非参数的,因为它们具有如此多的参数,以至于它们具有表征性容量过剩。

Discriminative vs. Generative



判别式模型是在寻找一个决策边界，通过该边界来将样本划分到对应类别。而生成式模型则不同，它学习了每个类别的边界，它包含了更多信息，可以用来生成样本。

target	descriptive feature	model
continuous		error based models
categorical	continuous	similarity based models
	categorical	information and probability models

项目要求

准确性

平均而言，集成模型和支持向量机是最精确的模型之一。

预测速度

逻辑回归模型非常快

k 最近邻模型非常慢

再培训能力

k 最近邻模型就是很好的例子

决策树模型是不好的例子

可解释性

决策树和线性回归模型很容易解释

支持向量机和深度学习网络几乎完全无法解释（黑匣子）

在选择一组初始方法时，需要考虑两个问题：

机器学习方法是否符合项目的要求？

该方法是否适合我们想要做出的预测类型以及我们正在使用的描述性特征类型？